Università Politecnica delle Marche

Corso di Laurea Triennale in Ingegneria Informatica e dell'Automazione

Progetto per il corso di Ingegneria del Software

Sistema di Gestione del Magazzino

Relatori:
Prof. Domenico Ursino
Prof. Davide Traini

Tesina di:
Califano Lorenzo
Carlomagno Chiara
Ferretti Sabrina

Anno Accademico 2024/2025

Contents

1 Descrizione del progetto **3**

2 Glossario dei Termini **4**

3 Requisiti **5**
3.1 Requisiti Funzionali 5
3.2 Requisiti Non Funzionali 7
3.3 Diagrammi dei casi d'Uso 8
3.3.1 Attori 8
3.3.2 Diagramma Gestione Utenti 9
3.3.3 Diagramma Gestione Prodotti 15
3.3.4 Diagramma Navigazione Prodotti 28
3.3.5 Diagramma Storico delle Operazioni 33
3.4 Matrice di Mapping 41

4 Analisi **42**
4.1 Diagrammi delle Classi 42
4.1.1 Package 42
4.1.2 User 43
4.1.3 Prodotto 44
4.1.4 Noleggio 45
4.1.5 Vendita 46
4.2 Diagrammi di sequenza 47
4.2.1 Login 47
4.2.2 Signup 48
4.2.3 Customer Singup 49
4.2.4 CUD 50
4.2.5 Registrazione Noleggio 51
4.2.6 Registrazione Vendita 52
4.2.7 Registrazione Prodotto Danneggiato 53
4.3 Diagrammi di attività 54
4.3.1 Login 54
4.3.2 Signup 55
4.3.3 Registrazione Noleggio 56
4.3.4 Registrazione Vendita 57
4.3.5 Registrazione Prodotto Danneggiato 58

5 Progettazione **59**
5.1 Classi di progettazione 59
5.1.1 Classi di progettazione: Package 59
5.1.2 Classi di progettazione: User 60
5.1.3 Classi di progettazione: Prodotto 61
5.1.4 Classi di progettazione: Noleggio 62
5.1.5 Classi di progettazione: Vendita 63
5.2 Diagramma dei componenti 64
5.3 Diagrammi delle macchine a stati 65

5.3.1 User 65
5.3.2 Prodotto 66
5.3.3 Noleggio 67
5.3.4 Vendita 68
5.4 Diagramma entità-relazioni 69

6 Implementazione **70**
6.1 Diagrammi di deployment 70
6.2 Tecnologie utilizzate 71
6.2.1 Tecnologie Front-End 71
6.2.2 Tecnologie Back-End 71
6.2.3 Tecnologie Cloud 72
6.2.4 Tecnologie di versionamento 72
6.2.5 Altre tecnologie utilizzate 72
6.3 Mockup 73
6.4 Descrizione unit tests 85

1 Descrizione del progetto

Il progetto riguarda lo sviluppo di un sistema gestionale per il magazzino di un'azienda che si occupa della vendita e del noleggio di attrezzature audio, video e luci, come microfoni, casse acustiche, mixer, ledwall, luci da discoteca, luminarie natalizie e strutture. Attualmente, la gestione del magazzino e dei noleggi avviene in modo manuale, senza un sistema centralizzato: i lavoratori tengono traccia delle attrezzature disponibili e dei noleggi senza un supporto digitale strutturato. La parte delle decorazioni natalizie, invece, è gestita dalla segreteria attraverso il software FattureInCloud. L'azienda ha una struttura operativa flessibile, in cui il personale si occupa direttamente delle operazioni di magazzino senza una suddivisione formale dei ruoli nella gestione delle attrezzature.

Il nuovo sistema gestionale avrà l'obiettivo di migliorare l'organizzazione del magazzino, consentendo il monitoraggio in tempo reale delle disponibilità, la gestione dei noleggi e delle vendite, e la segnalazione di danni. Un aspetto fondamentale sarà la gestione delle scorte, con notifiche automatiche per avvisare quando un prodotto è in esaurimento o fuori in noleggio. Il sistema dovrà anche garantire un controllo sulle modifiche effettuate, mantenendo uno storico delle operazioni per tracciare chi ha aggiornato le informazioni.

Gli utenti del sistema saranno suddivisi in tre ruoli principali. I magazzinieri avranno accesso alle funzioni operative del magazzino, potendo aggiornare le quantità di attrezzature disponibili, ricevere le notifiche e segnalare eventuali danni. La segreteria si occuperà della gestione amministrativa, con accesso alle informazioni di vendita e di noleggio e la possibilità di controllare le segnalazioni dei lavoratori. Infine, i responsabili aziendali avranno accesso completo al sistema, potendo monitorare l'attività complessiva, gestire gli utenti e consultare lo storico delle operazioni del magazzino e delle segnalazioni. Inoltre, avranno le stesse funzionalità operative dei lavoratori, potendo aggiornare le quantità di attrezzature disponibili, registrare i noleggi e segnalare eventuali danni o anomalie. Tutti gli utenti potranno visionare il catalogo

Ogni tipologia di attrezzatura sarà gestita in base alle sue varianti, come modello, dimensione o potenza, garantendo una corretta tracciabilità delle quantità disponibili. Per quanto riguarda i danni, i lavoratori potranno segnalarli direttamente nel sistema, ma le modifiche alle scorte dovranno essere approvate per garantire un controllo accurato. La registrazione di ogni segnalazione permetterà di mantenere uno storico utile per identificare eventuali problematiche ricorrenti su specifici prodotti o fornitori.

Anche se l'azienda al momento opera con un unico magazzino, il sistema sarà progettato per essere scalabile, in modo da poter gestire più sedi in futuro, se necessario. L'integrazione con un e-commerce o con strumenti avanzati di gestione degli ordini non è una priorità immediata, ma il sistema dovrà essere predisposto per eventuali sviluppi futuri.

L'obiettivo finale è creare un sistema semplice e intuitivo, che migliori l'efficienza operativa senza complicare le attività quotidiane dei lavoratori. Con una chiara separazione dei ruoli e un monitoraggio dettagliato delle operazioni, il nuovo gestionale permetterà all'azienda di avere un maggiore controllo sulle attrezzature, ottimizzando la gestione del magazzino e dei noleggi.

*In questo progetto, l'uso del genere maschile sovraesteso è dovuto unicamente a esigenze di semplicità del testo ed è da considerarsi inclusivo di tutta la Comunità.

2 Glossario dei Termini

Termine	Descrizione	Tipo	Sinonimi
Dipendente	Persona addetta alla gestione operativa del magazzino (es. aggiornare disponibilità, registrare noleggi).	Business	Magazziniere, Amministratore, Segretario, User
Noleggio	Processo di concessione temporanea di attrezzature a clienti.	Business	Affitto (contesto specifico: noleggio è preferito)
Segnalazione danni	Notifica inserita nel sistema per indicare un malfunzionamento o un danno a un'attrezzatura.	Business	Report anomalie, Segnalazione guasti
Storico operazioni	Registro digitale che traccia tutte le modifiche, noleggi, vendite e segnalazioni.	Tecnico	Log attività, Cronologia
Gestione scorte	Processo di monitoraggio e aggiornamento delle quantità disponibili in magazzino.	Business	Controllo inventario, Gestione inventario
Responsabile aziendale	Figura con accesso completo al sistema, abilitata a monitorare attività, gestire utenti e approvare modifiche critiche.	Business	Amministratore, Supervisore
Registrazione account	Creazione di un profilo utente per accedere al sistema, con permessi basati sul ruolo (magazziniere, segreteria, responsabile).	Tecnico	Sign up, Creazione profilo
Monitoraggio in tempo reale	Funzionalità che permette di visualizzare immediatamente le disponibilità e lo stato delle attrezzature.	Tecnico	Controllo live, Aggiornamento istantaneo
Login	Funzionalità che permette ad un Dipendente di accedere al proprio account utente.	Tecnico	Accesso
Tracciabilità	Capacità di seguire lo storico di un'attrezzatura, inclusi noleggi, danni e modifiche alle scorte.	Business	Rintracciamento, Cronologia prodotto
Catalogo	Elenco organizzato delle attrezzature disponibili per il noleggio.	Business	Inventario, Lista attrezzature

3 Requisiti

3.1 Requisiti Funzionali

Area: Gestione Utenti

RF1: Aggiungi Dipendente
Il sistema dovrà permettere ai responsabili di aggiungere un nuovo dipendente.

RF2: Login
Il sistema dovrà permettere al dipendente di accedere al proprio account.

RF3: Recupera Credenziali
Il sistema dovrà permettere al dipendente di recuperare le credenziali del proprio account.

RF4: Modifica Account
Il sistema dovrà permettere al dipendente di modificare le informazioni del proprio account.

RF5: Elimina Dipendente
Il sistema dovrà permettere ai responsabili di eliminare un dipendente esistente.

Area: Gestione Prodotti

RF6: Aggiungi Prodotto
Il sistema dovrà permettere al segretario di inserire un nuovo prodotto al catalogo.

RF7: Vendi Prodotto
Il sistema dovrà permettere al segretario di vendere un prodotto.

RF8: Noleggia Prodotto
Il sistema dovrà permettere al segretario di noleggiare un prodotto.

RF9: Modifica Prodotto
Il sistema dovrà permettere al segretario di modificare le informazioni di un prodotto.

RF10: Aggiungi Categoria
Il sistema dovrà permettere al segretario di inserire una nuova categoria di prodotti.

RF11: Rimuovi Prodotto
Il sistema dovrà permettere al segretario di rimuovere un prodotto dal catalogo.

RF12: Gestione Prodotti Danneggiati
Il sistema dovrà permettere al magazziniere di segnalare un danno ad un prodotto.

RF13: Aggiungi Potenza
Il sistema dovrà permettere al segretario di aggiungere una nuova potenza per i prodotti sul catalogo.

RF14: Aggiungi Modello
Il sistema dovrà permettere al segretario di aggiungere un nuovo modello per i prodotti sul catalogo.

RF15: Aggiungi Dimensione
Il sistema dovrà permettere al segretario di aggiungere una nuova dimensione per i prodotti sul catalogo.

RF16: Aggiungi Brand
Il sistema dovrà permettere al segretario di aggiungere un nuovo brand per i prodotti sul catalogo.

RF17: Modifica Quantità
Il sistema dovrà permettere al magazziniere di modificare la quantità di un prodotto.

Area: Navigazione Prodotti

RF18: Visualizza Prodotto
Il sistema dovrà mostrare le informazioni dettagliate di un prodotto.

RF19: Visualizza Catalogo
Il sistema dovrà mostrare al dipendente i prodotti presenti nel catalogo.

RF20: Cerca Prodotto
Il sistema dovrà permettere al dipendente di cercare il prodotto desiderato tramite ricerca testuale.

RF21: Filtra Prodotti
Il sistema dovrà permettere al dipendente di filtrare i prodotti tramite filtri per potenza, modello, dimensione o brand.

Area: Storico delle Operazioni

RF22: Aggiungi Noleggio
Il sistema dovrà permettere al segretario di notificare un noleggio a tutti i dipendenti.

RF23: Aggiungi Vendita
Il sistema dovrà permettere al segretario di notificare una vendita a tutti i dipendenti.

RF24: Modifica Noleggio
Il sistema dovrà permettere al segretario di notificare una modifica di un noleggio a tutti i dipendenti.

RF25: Annulla Noleggio
Il sistema dovrà permettere al segretario di notificare l'annullamento di un noleggio a tutti i dipendenti.

RF26: Concludi Noleggio
Il sistema dovrà permettere al segretario di notificare l'avvenuta conclusione di un noleggio a tutti i dipendenti.

RF27: Annulla Vendita
Il sistema dovrà permettere al segretario di notificare l'annullamento di una vendita a tutti i dipendenti.

RF28: Aggiungi Danno
Il sistema dovrà permettere al magazziniere di notificare l'aggiunta di un danno a tutti i dipendenti.

3.2 Requisiti Non Funzionali

Area: Gestione Tecnologie e vincoli

RNF1: Implementazione in Python3.11.9
Il sistema dovrà essere implementanto in linguaggio Python3.11.9

RNF2: Utilizzo di un Database Relazionale
Il sistema dovrà adoperare un database relazionale per il salvataggio dei dati.

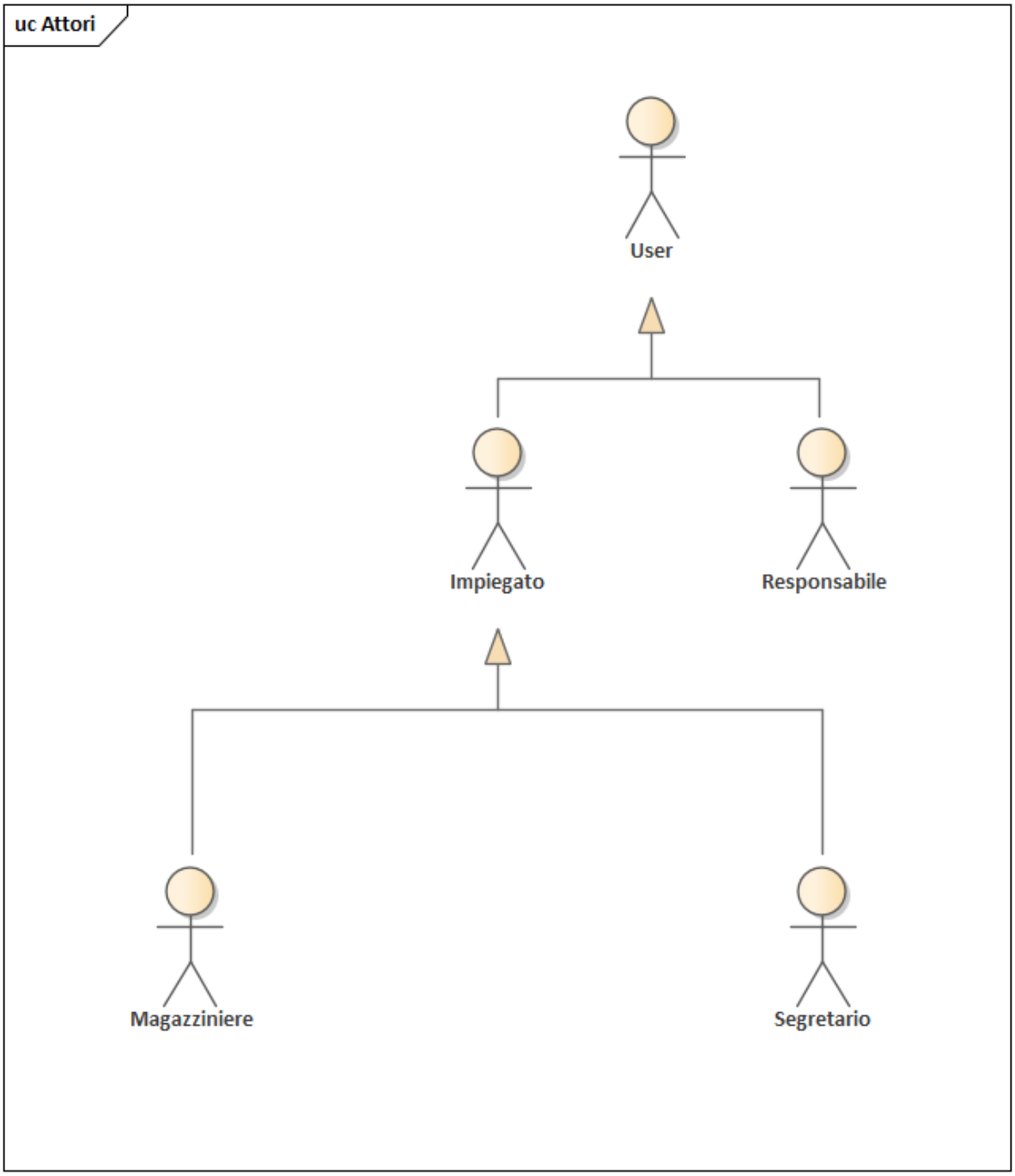
RNF3: Verifica Email
Il sistema dovrà verificare, durante la registrazione, che l'utente è il proprietario dell'email inserita.

RNF4: Notifica quantità Esaurita o in Esaurimento
Il sistema dovrà notificare tutti i dipendenti se la quantità di un prodotto è in esaurimento o è esaurita.

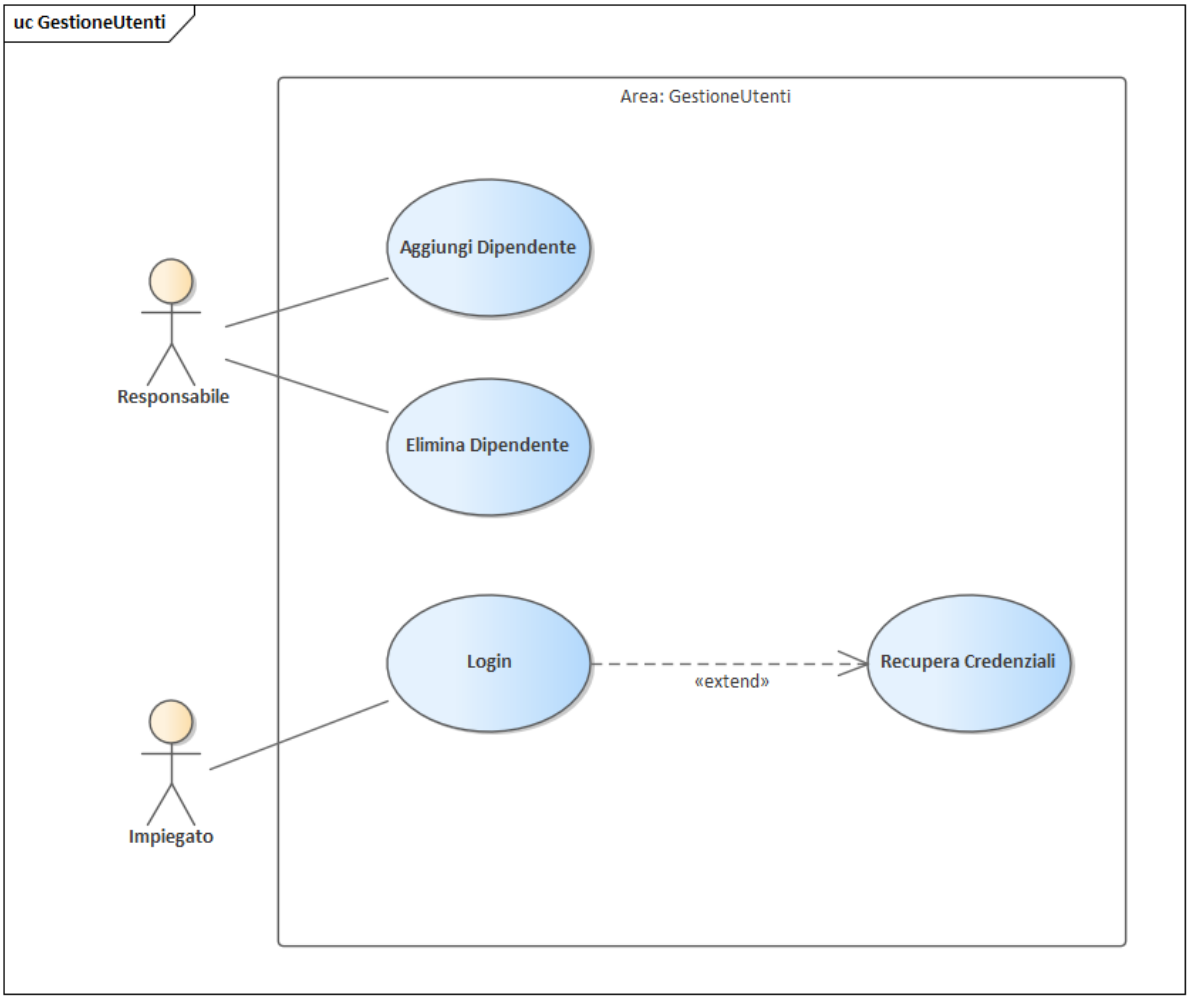
RNF5: Notifica Modifica Prodotto
Il sistema dovrà notificare tutti i dipendenti se viene modificato un prodotto.

3.3 Diagrammi dei casi d'Uso

3.3.1 Attori



3.3.2 Diagramma Gestione Utenti



Caso d'uso: AggiungiDipendente

Caso d'uso: AggiungiDipendente
ID: 1
Breve Descrizione: Il Responsabile aggiunge un nuovo dipendente al sistema.
Attori Primari: Responsabile.
Attori Secondari: Nessuno.
Precondizioni: 1. Il Responsabile ha effettuato il login.
Sequenza degli eventi principale: 1. Il caso d'uso inizia selezionando "Aggiungi Dipendente". 2. Il sistema richiede: nome, cognome, email, ruolo. 3. Il Responsabile inserisce i dati. 4. Il sistema verifica l'unicità dell'email. 5. Il sistema crea l'account e invia credenziali.
Postcondizioni: 1. Nuovo account creato.
Sequenza degli eventi alternativa: 4a. Se email esistente, sistema mostra errore.

Caso d'Uso: Login

Caso d'uso: Login
ID: 2
Breve Descrizione: Accesso al Sistema.
Attori Primari: Dipendente.
Attori Secondari: Nessuno.
Precondizioni: 1. L'account esiste.
Sequenza degli eventi principale: 1. Il caso d'uso inizia selezionando "Login". 2. Il sistema richiede: email e password. 3. Il Dipendente inserisce le credenziali. 4. Il sistema verifica l'autenticità.
Postcondizioni: 1. Accesso consentito.
Sequenza degli eventi alternativa: 4a. Credenziali errate: Il Sistema mostra un errore.

Caso d'Uso: RecuperaCredenziali

Caso d'uso: RecuperaCredenziali
ID: 3
Breve Descrizione: Recupero credenziali dimenticate.
Attori Primari: Dipendente.
Attori Secondari: Nessuno.
Precondizioni: 1. L'account esiste.
Sequenza degli eventi principale: 1. Il caso d'uso inizia selezionando "Recupera Credenziali". 2. Il sistema richiede: email. 3. Il Dipendente inserisce la mail. 4. Il sistema verifica la mail. 5. Il sistema invia la nuova password al dipendente.
Postcondizioni: 1. Email di recupero inviata.
Sequenza degli eventi alternativa: 4a. Email non trovata: Il Sistema mostra un errore.

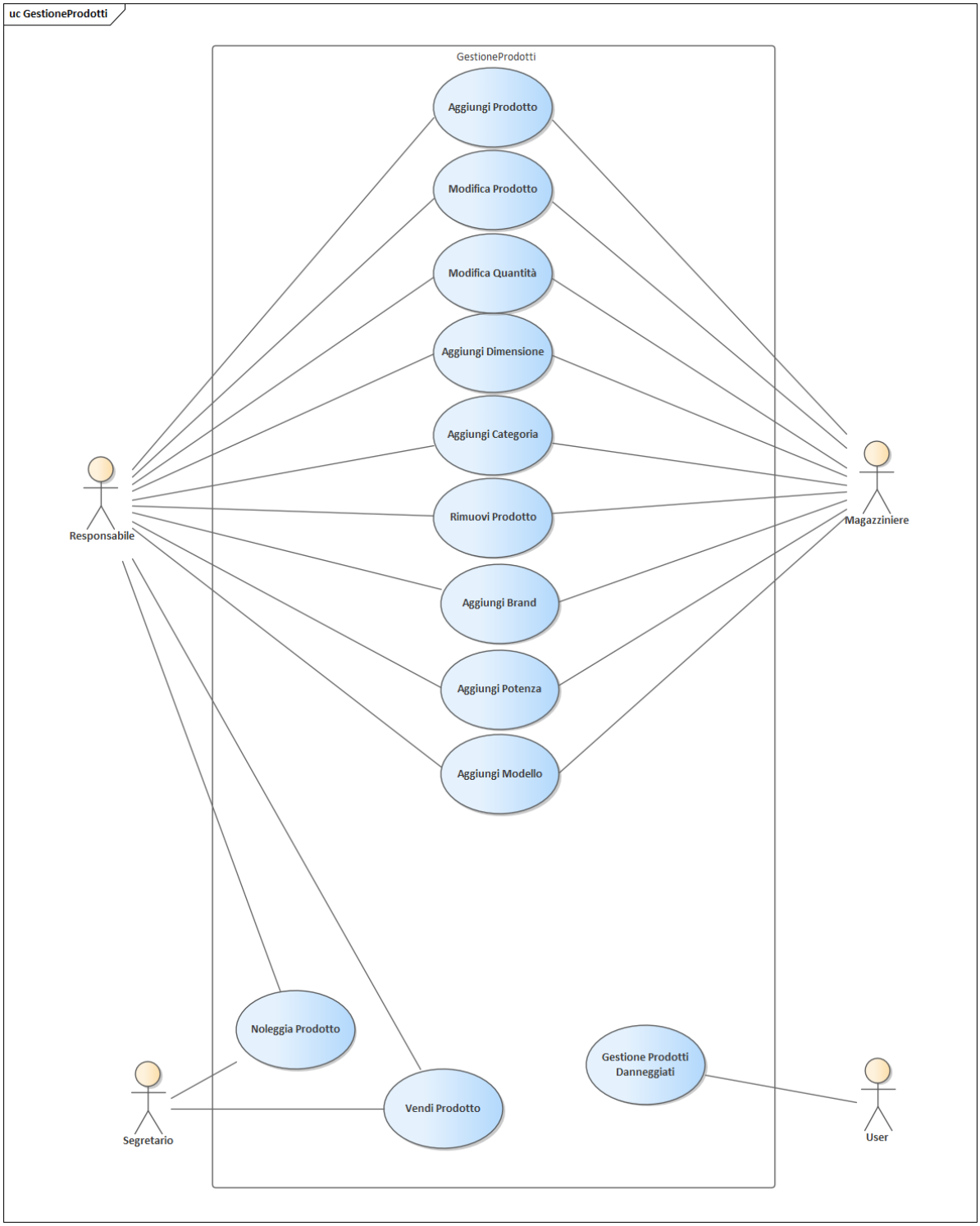
Caso d'Uso: ModificaAccount

Caso d'uso: ModificaAccount
ID: 4
Breve Descrizione: Aggiornamento informazioni account.
Attori Primari: Dipendente.
Attori Secondari: Nessuno.
Precondizioni: 1. Il Dipendente ha effettuato il login.
Sequenza degli eventi principale: 1. Il caso d'uso inizia selezionando "Modifica Account". 2. Il sistema mostra i dati correnti. 3. Il Dipendente modifica i campi. 4. Il sistema salva le modifiche.
Postcondizioni: 1. Account aggiornato.
Sequenza degli eventi alternativa: Nessuna.

Caso d'Uso: EliminaDipendente

Caso d'uso: EliminaDipendente
ID: 5
Breve Descrizione: Rimozione di un Dipendente dal sistema.
Attori Primari: Responsabile
Attori Secondari: Nessuno.
Precondizioni: 1. Il Responsabile ha effettuato il login. 2. Il Dipendente esiste.
Sequenza degli eventi principale: 1. Il caso d'uso inizia selezionando "Gestione Dipendenti". 2. Il sistema mostra l'elenco dei Dipendenti. 3. Il Responsabile seleziona il testo elimina di un Dipendente. 4. Il sistema conferma l'eliminazione.
Postcondizioni: 1. Account rimosso.
Sequenza degli eventi alternativa: 4a. Operazione annullata: nessuna modifica.

3.3.3 Diagramma Gestione Prodotti



Caso d'Uso: AggiungiProdotto

Caso d'uso: AggiungiProdotto
ID: 6
Breve Descrizione: Inserimento di un nuovo prodotto nel catalogo.
Attori Primari: Segretario.
Attori Secondari: Nessuno.
Precondizioni: 1. Il Segretario ha effettuato il login.
Sequenza degli eventi principale: 1. Il caso d'uso inizia selezionando "Aggiungi Prodotto". 2. Il sistema richiede: nome, categoria, quantità, varianti. 3. Il Segeretario inserisce i dati. 4. Il sistema aggiunge il prodotto.
Postcondizioni: 1. Il nuovo prodotto è disponibile nel catalogo.
Sequenza degli eventi alternativa: Nessuna.

Caso d'Uso: VendiProdotto

Caso d'uso: VendiProdotto
ID: 7
Breve Descrizione: Registrazione della vendita di un prodotto.
Attori Primari: Segretario.
Attori Secondari: Nessuno.
Precondizioni: 1. Il Segretario ha effettuato il login. 2. Il prodotto è disponibile.
Sequenza degli eventi principale: 1. Il caso d'uso inizia selezionando "Vendi Prodotto". 2. Il sistema richiede: prodotto, quantità, nome dell'acquirente e metodo di pagamento. 3. Il Segeretario conferma la vendita. 4. Il sistema aggiorna le scorte.
Postcondizioni: 1. La vendita è registrata. 2. Le scorte sono aggiornate
Sequenza degli eventi alternativa: Nessuna.

Caso d'Uso: NoleggiaProdotto

Caso d'uso: NoleggiaProdotto
ID: 8
Breve Descrizione: Registrazione del noleggio di un prodotto.
Attori Primari: Segretario.
Attori Secondari: Nessuno.
Precondizioni: 1. Il Segretario ha effettuato il login. 2. Il prodotto è disponibile.
Sequenza degli eventi principale: 1. Il caso d'uso inizia selezionando "Noleggia Prodotto". 2. Il sistema richiede: prodotto, quantità, nome del noleggiante, data di fine noleggio e metodo di pagamento. 3. Il Segeretario inserisce i dati. 4. Il sistema registra il noleggio.
Postcondizioni: 1. Il noleggio è registrato. 2. Le scorte sono aggiornate.
Sequenza degli eventi alternativa: Nessuna.

Caso d'Uso: ModificaProdotto

Caso d'uso: ModificaProdotto
ID: 9
Breve Descrizione: Aggiornamento delle informazioni di un prodotto.
Attori Primari: Segretario.
Attori Secondari: Nessuno.
Precondizioni: 1. Il Segretario ha effettuato il login. 2. Il prodotto è esistente
Sequenza degli eventi principale: 1. Il caso d'uso inizia selezionando "Modifica Prodotto". 2. Il sistema mostra i dati correnti. 3. Il Segeretario modifica i campi. 4. Il sistema salva le modifiche.
Postcondizioni: 1. Il prodotto è aggiornato.
Sequenza degli eventi alternativa: Nessuna.

Caso d'Uso: AggiungiCategoria

Caso d'uso: AggiungiCategoria
ID: 10
Breve Descrizione: Creazione di una nuova categoria di prodotti.
Attori Primari: Segretario.
Attori Secondari: Nessuno.
Precondizioni: 1. Il Segretario ha effettuato il login.
Sequenza degli eventi principale: 1. Il caso d'uso inizia aggiungendo un prodotto al catalogo. 2. Il sistema richiede il nome della categoria e le informazioni del prodotto 3. Il Segeretario inserisce il nome della categoria. 3.1 Se la categoria esiste il prodotto viene aggiunto al catalogo. 3.2 Se la categoria non esiste viene creata una nuova categoria e poi il prodotto viene aggiunto al catalogo.
Postcondizioni: 1. È disponibile una nuova categoria.
Sequenza degli eventi alternativa: Nessuna.

Caso d'Uso: RimuoviProdotto

Caso d'uso: RimuoviProdotto
ID: 11
Breve Descrizione: Rimozione di un prodotto dal catalogo.
Attori Primari: Segretario.
Attori Secondari: Nessuno.
Precondizioni: 1. Il Segretario ha effettuato il login. 2. Il prodotto è esistente.
Sequenza degli eventi principale: 1. Il caso d'uso inizia selezionando "Rimuovi Prodotto". 2. Il mostra l'elenco dei prodotti. 3. Il Segeretario seleziona un prodotto. 4. Il sistema conferma la rimozione.
Postcondizioni: 1. Il prodotto è stato rimosso dal catalogo.
Sequenza degli eventi alternativa: 4a. Operazione annullata: nessuna modifica.

Caso d'Uso: SegnalaDanno

Caso d'uso: SegnalaDanno
ID: 12
Breve Descrizione: Segnalazione di una danno ad un prodotto.
Attori Primari: Magazziniere.
Attori Secondari: Nessuno.
Precondizioni: 1. Il Magazziniere ha effettuato il login. 2. Il prodotto è esistente
Sequenza degli eventi principale: 1. Il caso d'uso inizia selezionando "Segnala Danno". 2. Il sistema richiede: prodotto, descrizione del danno. 3. Il Magazziniere inserisce i dati. 4. Il sistema registra la segnalazione.
Postcondizioni: 1. Il danno è registrato nel sistema.
Sequenza degli eventi alternativa: Nessuna.

Caso d'Uso: AggiungiPotenza

Caso d'uso: AggiungiPotenza
ID: 13
Breve Descrizione: Aggiunta di una nuova potenza per i prodotti.
Attori Primari: Segretario.
Attori Secondari: Nessuno.
Precondizioni: 1. Il Segretario ha effettuato il login.
Sequenza degli eventi principale: 1. Il caso d'uso inizia aggiungendo un prodotto al catalogo. 2. Il sistema richiede il valore della potenza e le informazioni del prodotto 3. Il Segeretario inserisce il valore della potenza. 3.1 Se il valore esiste il prodotto viene aggiunto al catalogo. 3.2 Se il valore non esiste viene creata un nuovo valore potenza e poi il prodotto viene aggiunto al catalogo.
Postcondizioni: 1. La nuova potenza è disponibile.
Sequenza degli eventi alternativa: Nessuna.

Caso d'Uso: AggiungiModello

Caso d'uso: AggiungiModello
ID: 14
Breve Descrizione: Aggiunta di un nuovo modello per i prodotti.
Attori Primari: Segretario.
Attori Secondari: Nessuno.
Precondizioni: 1. Il Segretario ha effettuato il login.
Sequenza degli eventi principale: 1. Il caso d'uso inizia aggiungendo un prodotto al catalogo. 2. Il sistema richiede il nome del modello e le informazioni del prodotto 3. Il Segeretario inserisce il nome del modello. 3.1 Se il modello esiste il prodotto viene aggiunto al catalogo. 3.2 Se il modello non esiste viene creata un nuovo modello e poi il prodotto viene aggiunto al catalogo.
Postcondizioni: 1. Il nuovo modello è disponibile.
Sequenza degli eventi alternativa: Nessuna.

Caso d'Uso: AggiungiDimensione

Caso d'uso: AggiungiDimensione
ID: 15
Breve Descrizione: Aggiunta di una nuova dimensione per i prodotti.
Attori Primari: Segretario.
Attori Secondari: Nessuno.
Precondizioni: 1. Il Segretario ha effettuato il login.
Sequenza degli eventi principale: 1. Il caso d'uso inizia aggiungendo un prodotto al catalogo. 2. Il sistema richiede la dimensione e le informazioni del prodotto 3. Il Segeretario inserisce la dimensione. 3.1 Se la dimensione esiste il prodotto viene aggiunto al catalogo. 3.2 Se la dimensione non esiste viene creata una nuova dimensione e poi il prodotto viene aggiunto al catalogo.
Postcondizioni: 1. La nuova dimensione è disponibile.
Sequenza degli eventi alternativa: Nessuna.

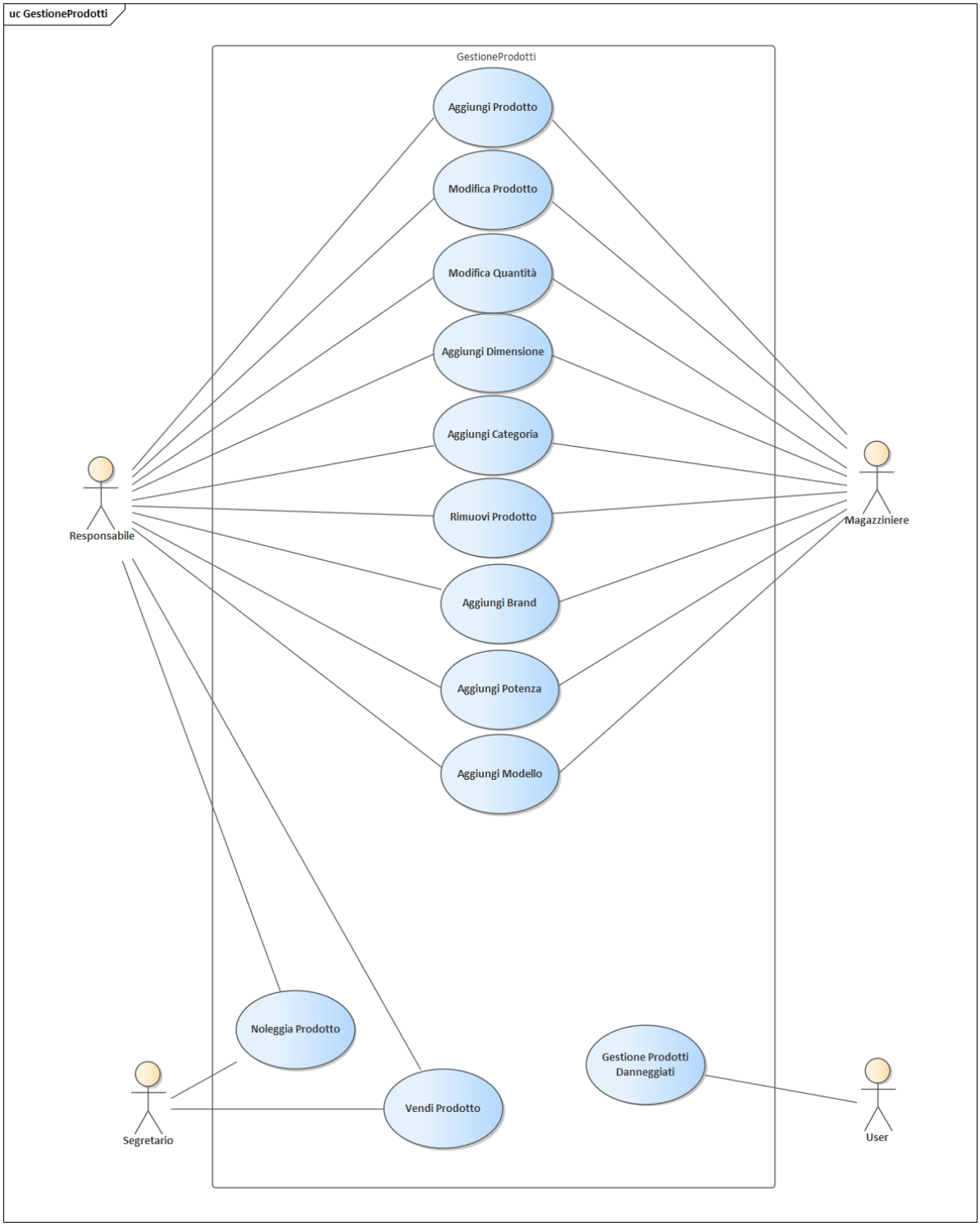
Caso d'Uso: AggiungiBrand

Caso d'uso: AggiungiBrand
ID: 16
Breve Descrizione: Aggiunta di un nuovo brand per i prodotti.
Attori Primari: Segretario.
Attori Secondari: Nessuno.
Precondizioni: 1. Il Segretario ha effettuato il login.
Sequenza degli eventi principale: 1. Il caso d'uso inizia aggiungendo un prodotto al catalogo. 2. Il sistema richiede il nome del brand e le informazioni del prodotto 3. Il Segeretario inserisce il nome del brand. 3.1 Se il brand esiste il prodotto viene aggiunto al catalogo. 3.2 Se il brand non esiste viene creata un nuovo brand e poi il prodotto viene aggiunto al catalogo.
Postcondizioni: 1. Il nuovo brand è disponibile.
Sequenza degli eventi alternativa: 3a. Brand già esistente: Il sistema mostra un errore.

Caso d'Uso: ModificaQuantità

Caso d'uso: ModificaQuantità
ID: 17
Breve Descrizione: Aggiornamento della quantità di un prodotto.
Attori Primari: Magazziniere.
Attori Secondari: Nessuno.
Precondizioni: 1. Il Magazziniere ha effettuato il login. 2. Il prodotto è esistente.
Sequenza degli eventi principale: 1. Il caso d'uso inizia selezionando "Modifica quantità". 2. Il sistema mostra la quantità corrente. 3. Il Magazziniere inserisce la nuova quantità. 4. Il sistema aggiorna le scorte.
Postcondizioni: 1. La quantità del prodotto è aggiornata.
Sequenza degli eventi alternativa: Nessuna.

3.3.4 Diagramma Navigazione Prodotti



Caso d'Uso: VisualizzaProdotto

Caso d'uso: VisualizzaProdotto
ID: 18
Breve Descrizione: Visualizzazione dei dettagli di un prodotto.
Attori Primari: Dipendente.
Attori Secondari: Nessuno.
Precondizioni: 1. Il Dipendente ha effettuato il login.
Sequenza degli eventi principale: 1. Il caso d'uso inizia selezionando un prodotto. 2. Il sistema recupera le informazioni. 3. Il sistema mostra: nome, categoria, quantità, varianti.
Postcondizioni: Nessuna.
Sequenza degli eventi alternativa: Nessuna.

Caso d'Uso: VisualizzaCatalogo

Caso d'uso: VisualizzaCatalogo
ID: 19
Breve Descrizione: Visualizzazione del catalogo completo.
Attori Primari: Dipendente.
Attori Secondari: Nessuno.
Precondizioni: 1. Il Dipendente ha effettuato il login.
Sequenza degli eventi principale: 1. Il caso d'uso inizia selezionando "Catalogo". 2. Il sistema recupera tutti i prodotti. 3. Il sistema mostra l'elenco dei prodotti con le informazioni essenziali.
Postcondizioni: Nessuna.
Sequenza degli eventi alternativa: 2a. Catalogo vuoto: In assenza di prodotti il sistema mostra una schermata vuota.

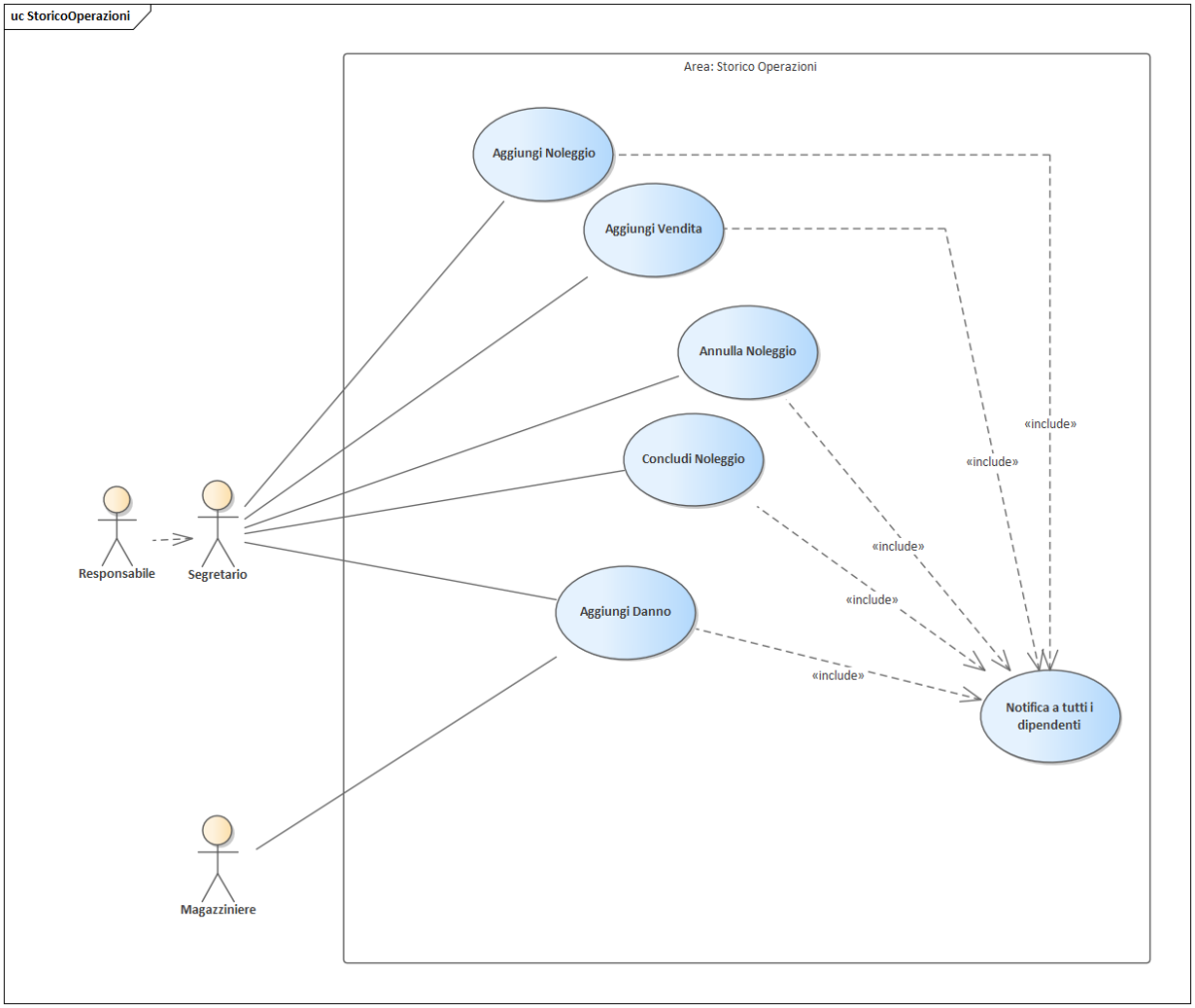
Caso d'Uso: CercaProdotto

Caso d'uso: CercaProdotto
ID: 20
Breve Descrizione: Ricerca dei prodotti per testo.
Attori Primari: Dipendente.
Attori Secondari: Nessuno.
Precondizioni: 1. Il Dipendente ha effettuato il login.
Sequenza degli eventi principale: 1. Il caso d'uso inizia inserendo il testo da ricercare. 2. Il sistema cerca corrispondenze in nome. 3. Il sistema mostra i risultati.
Postcondizioni: Nessuna.
Sequenza degli eventi alternativa: 2a. Nessun risultato: In assenza di prodotti il sistema mostra una schermata vuota.

Caso d'Uso: FiltraProdotti

Caso d'uso: FiltraProdotti
ID: 21
Breve Descrizione: Filtraggio dei prodotti per attributi.
Attori Primari: Dipendente.
Attori Secondari: Nessuno.
Precondizioni: 1. Il Dipendente ha effettuato il login.
Sequenza degli eventi principale: 1. Il caso d'uso inizia selezionando "Catalogo". 2. Il sistema richiede i parametri (categoria/modello/dimensione/brand). 3. Il Dipendente seleziona i filtri. 4. Il sistema mostra i prodotti corrispondenti.
Postcondizioni: Nessuna.
Sequenza degli eventi alternativa: 4a. Nessun risultato: In assenza di prodotti il sistema mostra una schermata vuota.

3.3.5 Diagramma Storico delle Operazioni



Caso d'Uso: AggiungiNoleggio

Caso d'uso: AggiungiNoleggio
ID: 22
Breve Descrizione: Notifica di un nuovo noleggio a tutti i Dipendenti.
Attori Primari: Segretario.
Attori Secondari: Nessuno.
Precondizioni: 1. Il Segretario ha effettuato il login.
Sequenza degli eventi principale: 1. Il caso d'uso inizia dopo la registrazione di un noleggio. 2. Il sistema genera una notifica. 3. Il sistema invia la notifica a tutti i Dipendenti.
Postcondizioni: 1. I Dipendenti ricevono la notifica.
Sequenza degli eventi alternativa: Nessuna.

Caso d'Uso: AggiungiVendita

Caso d'uso: AggiungiVendita
ID: 23
Breve Descrizione: Notifica di una nuova vendita a tutti i Dipendenti.
Attori Primari: Segretario.
Attori Secondari: Nessuno.
Precondizioni: 1. Il Segretario ha effettuato il login.
Sequenza degli eventi principale: 1. Il caso d'uso inizia dopo la registrazione di una vendita. 2. Il sistema genera una notifica. 3. Il sistema invia la notifica a tutti i Dipendenti.
Postcondizioni: 1. I Dipendenti ricevono la notifica.
Sequenza degli eventi alternativa: Nessuna.

Caso d'Uso: ModificaNoleggio

Caso d'uso: ModificaNoleggio
ID: 24
Breve Descrizione: Notifica di una modifica di noleggio a tutti i Dipendenti.
Attori Primari: Segretario.
Attori Secondari: Nessuno.
Precondizioni: 1. Il Segretario ha effettuato il login. 2. Il noleggio è esistente.
Sequenza degli eventi principale: 1. Il caso d'uso inizia dopo la modifica di un noleggio. 2. Il sistema genera una notifica. 3. Il sistema invia la notifica a tutti i Dipendenti.
Postcondizioni: 1. I Dipendenti ricevono la notifica.
Sequenza degli eventi alternativa: Nessuna.

Caso d'Uso: AnnullaNoleggio

Caso d'uso: AnnullaNoleggio
ID: 25
Breve Descrizione: Notifica dell'annullamento di un noleggio a tutti i Dipendenti.
Attori Primari: Segretario.
Attori Secondari: Nessuno.
Precondizioni: 1. Il Segretario ha effettuato il login. 2. Il noleggio è esistente.
Sequenza degli eventi principale: 1. Il caso d'uso inizia dopo l'annullamento di un noleggio. 2. Il sistema genera una notifica. 3. Il sistema invia la notifica a tutti i Dipendenti.
Postcondizioni: 1. I Dipendenti ricevono la notifica.
Sequenza degli eventi alternativa: Nessuna.

Caso d'Uso: ConcludiNoleggio

Caso d'uso: ConcludiNoleggio
ID: 26
Breve Descrizione: Notifica della conclusione di un noleggio a tutti i Dipendenti.
Attori Primari: Segretario.
Attori Secondari: Nessuno.
Precondizioni: 1. Il Segretario ha effettuato il login. 2. Il noleggio è esistente.
Sequenza degli eventi principale: 1. Il caso d'uso inizia dopo la restituzione del prodotto noleggiato. 2. Il sistema genera una notifica. 3. Il sistema invia la notifica a tutti i Dipendenti.
Postcondizioni: 1. I Dipendenti ricevono la notifica..
Sequenza degli eventi alternativa: Nessuna.

Caso d'Uso: AnnullaVendita

Caso d'uso: AnnullaVendita
ID: 27
Breve Descrizione: Notifica dell'annullamento di una vendita a tutti i Dipendenti.
Attori Primari: Segretario.
Attori Secondari: Nessuno
Precondizioni: 1. Il Segretario ha effettuato il login. 2. La vendita è esistente.
Sequenza degli eventi principale: 1. Il caso d'uso inizia dopo l'annullamento di una vendita. 2. Il sistema genera una notifica. 3. Il sistema invia la notifica a tutti i Dipendenti.
Postcondizioni: 1. I Dipendenti ricevono la notifica..
Sequenza degli eventi alternativa: Nessuna.

Caso d'Uso: AggiungiDanno

Caso d'uso: AggiungiDanno
ID: 28
Breve Descrizione: Notifica la segnalazione di un danno a tutti i Dipendenti.
Attori Primari: Magazziniere.
Attori Secondari: Nessuno
Precondizioni: 1. Il Magazziniere ha effettuato il login. 2. La segnalazione è stata effettuata.
Sequenza degli eventi principale: 1. Il caso d'uso inizia dopo la segnalazione di un danno. 2. Il sistema genera una notifica. 3. Il sistema invia la notifica a tutti i Dipendenti.
Postcondizioni: 1. I Dipendenti ricevono la notifica.
Sequenza degli eventi alternativa: Nessuna.

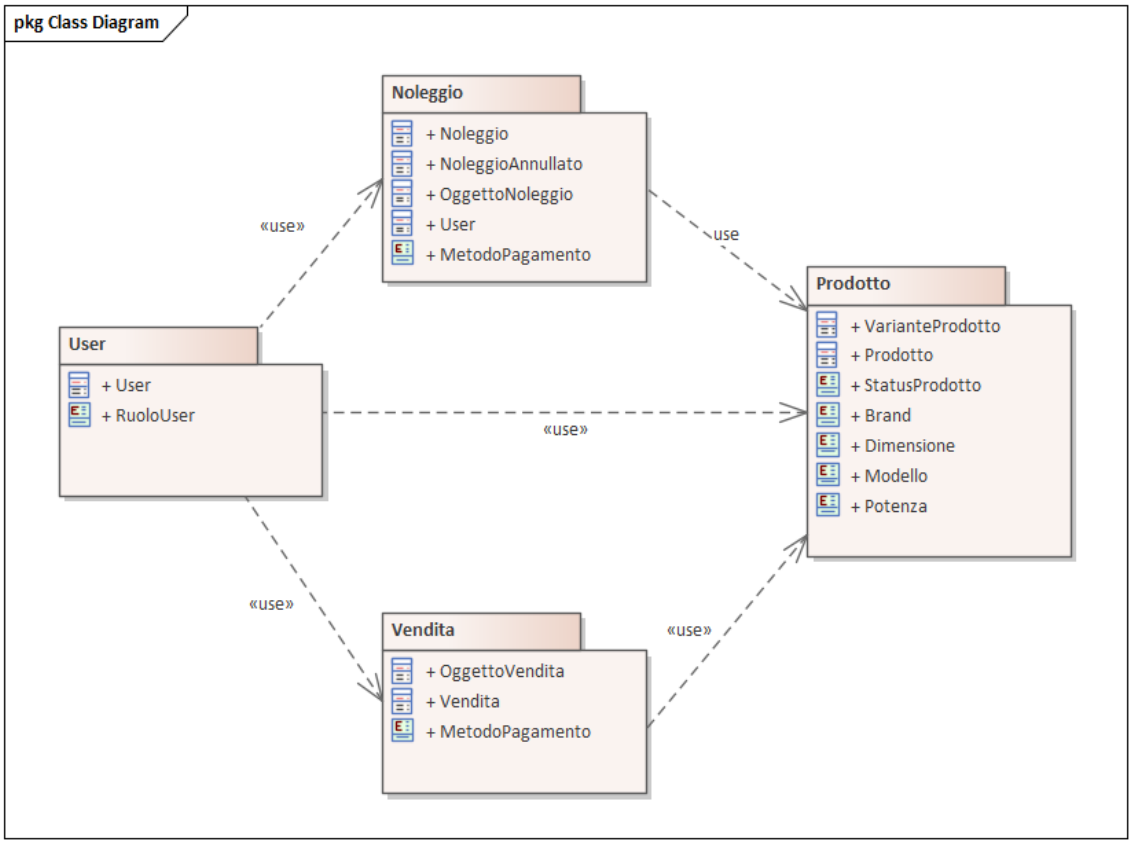
3.4 Matrice di Mapping

Source \ Target	GestioneProdotti::Affitta Prodotto	GestioneProdotti::Aggiungi Brand	GestioneProdotti::Aggiungi Categoria	GestioneProdotti::Aggiungi Dimensione	GestioneProdotti::Aggiungi Modello	GestioneProdotti::Aggiungi Potenza	GestioneProdotti::Aggiungi prodotto	GestioneProdotti::Elimina Prodotto	GestioneProdotti::Gestione Prodotto Danneggiato	GestioneProdotti::Modifica Prodotto	GestioneProdotti::Modifica Quantità	GestioneProdotti::Vendi prodotto	GestioneUtenti::Aggiungi Dipendente	GestioneUtenti::Elimina Dipendente	GestioneUtenti::Login	GestioneUtenti::Modifica Account	GestioneUtenti::Recupera Credenziali	NavigazioneProdotti::Cerca Prodotto	NavigazioneProdotti::Filtra Categoria	NavigazioneProdotti::Filtra Prodotti	NavigazioneProdotti::Visualizza Catalogo	NavigazioneProdotti::Visualizza Prodotto	StoricoOperazioni::Aggiungi Danno	StoricoOperazioni::Aggiungi Noleggio	StoricoOperazioni::Aggiungi Vendita	StoricoOperazioni::Annulla Noleggio	StoricoOperazioni::Annulla Vendita	StoricoOperazioni::Elimina Noleggio	StoricoOperazioni::Modifica Noleggio	StoricoOperazioni::Notifica a tutti i dipendenti
Casi D'Uso::Affitta Prodotto	↑																												
Casi D'Uso::Aggiungi Brand		↑																											
Casi D'Uso::Aggiungi Categoria			↑																										
Casi D'Uso::Aggiungi Danno																						↑							
Casi D'Uso::Aggiungi Dimensione				↑																									
Casi D'Uso::Aggiungi Dipendente													↑																
Casi D'Uso::Aggiungi Modello					↑																								
Casi D'Uso::Aggiungi Noleggio																							↑						
Casi D'Uso::Aggiungi Potenza						↑																							
Casi D'Uso::Aggiungi Prodotto							↑																						
Casi D'Uso::Aggiungi Vendita																								↑					
Casi D'Uso::Annulla Noleggio																									↑				
Casi D'Uso::Annulla Vendita																										↑			
Casi D'Uso::Cerca Prodotto																		↑											
Casi D'Uso::Concludi Noleggio																											↑		
Casi D'Uso::Elimina Dipendente														↑															
Casi D'Uso::Filtra Categoria																			↑										
Casi D'Uso::Filtra Prodotti																				↑									
Casi D'Uso::Gestione Prodotti Danneggiati									↑																				
Casi D'Uso::Login															↑														
Casi D'Uso::Modifica Account																↑													
Casi D'Uso::Modifica Noleggio																												↑	
Casi D'Uso::Modifica Prodotto										↑																			
Casi D'Uso::Modifica Quantità											↑																		
Casi D'Uso::Notifica a tutti i dipendenti																													↑
Casi D'Uso::Recupera Credenziali																	↑												
Casi D'Uso::Rimuovi Prodotto								↑																					
Casi D'Uso::Vendi Prodotto												↑																	
Casi D'Uso::Visualizza Catalogo																					↑								
Casi D'Uso::Visualizza Prodotto																						↑							

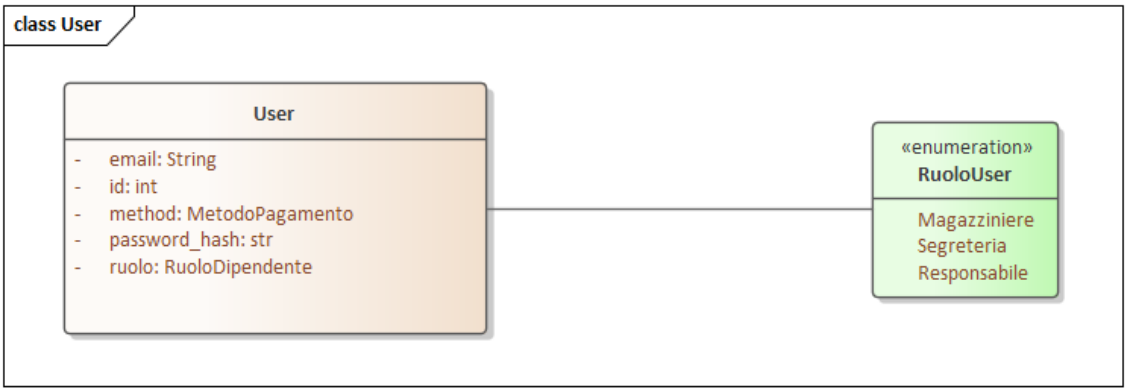
4 Analisi

4.1 Diagrammi delle Classi

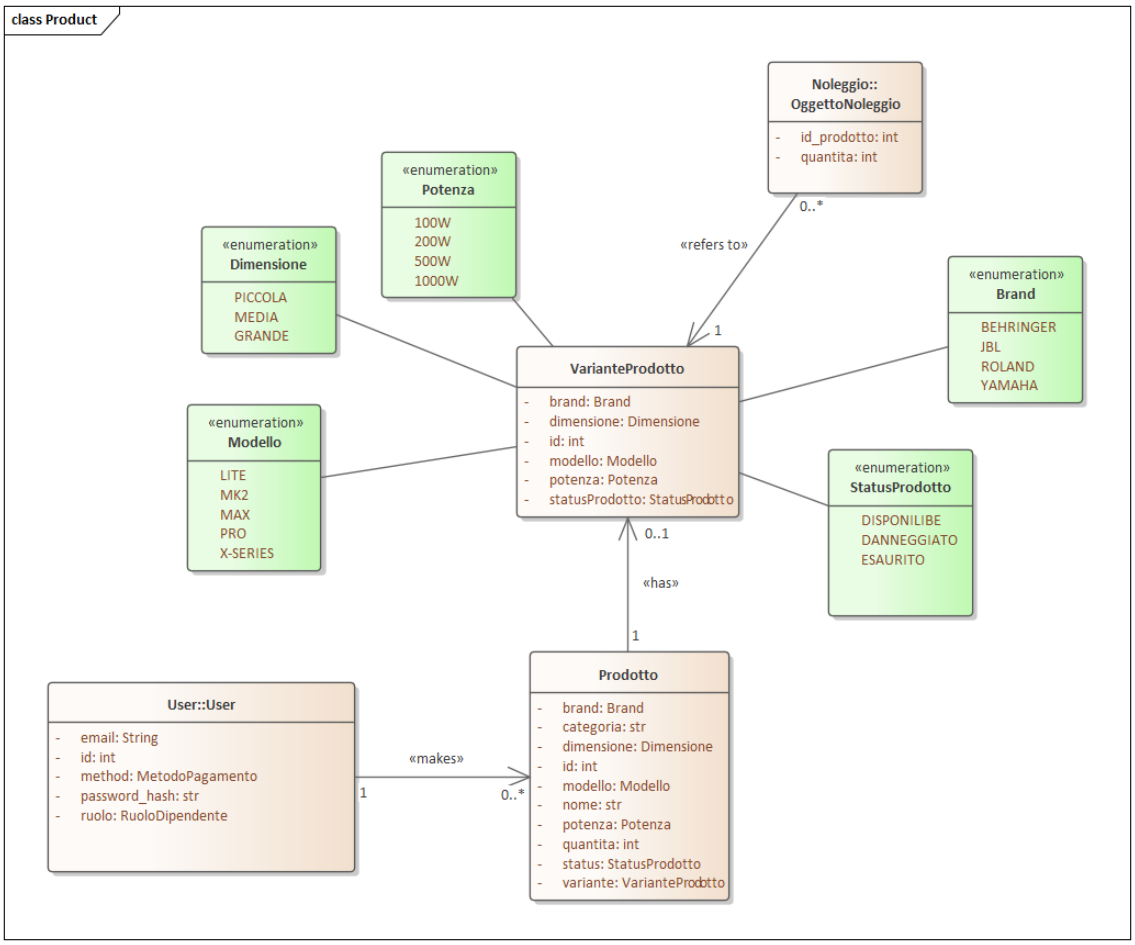
4.1.1 Package



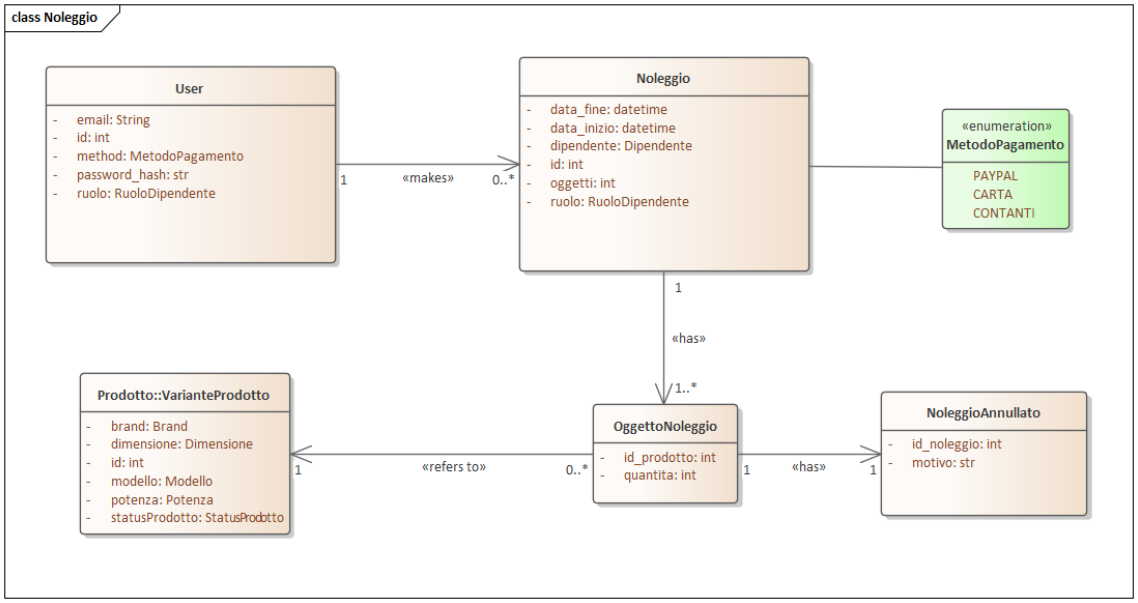
4.1.2 User


class User
User
- email: String
- id: int
- method: MetodoPagamento
- password_hash: str
- ruolo: RuoloDipendente
«enumeration»
RuoloUser
Magazziniere
Segreteria
Responsabile

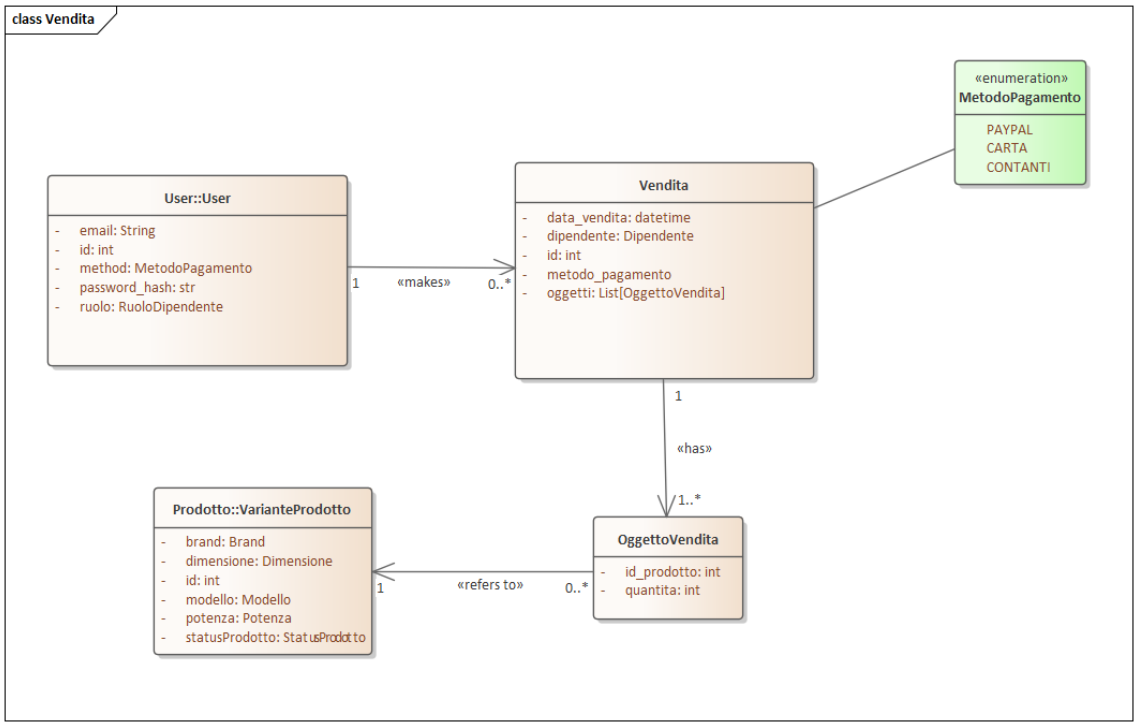
4.1.3 Prodotto



4.1.4 Noleggio

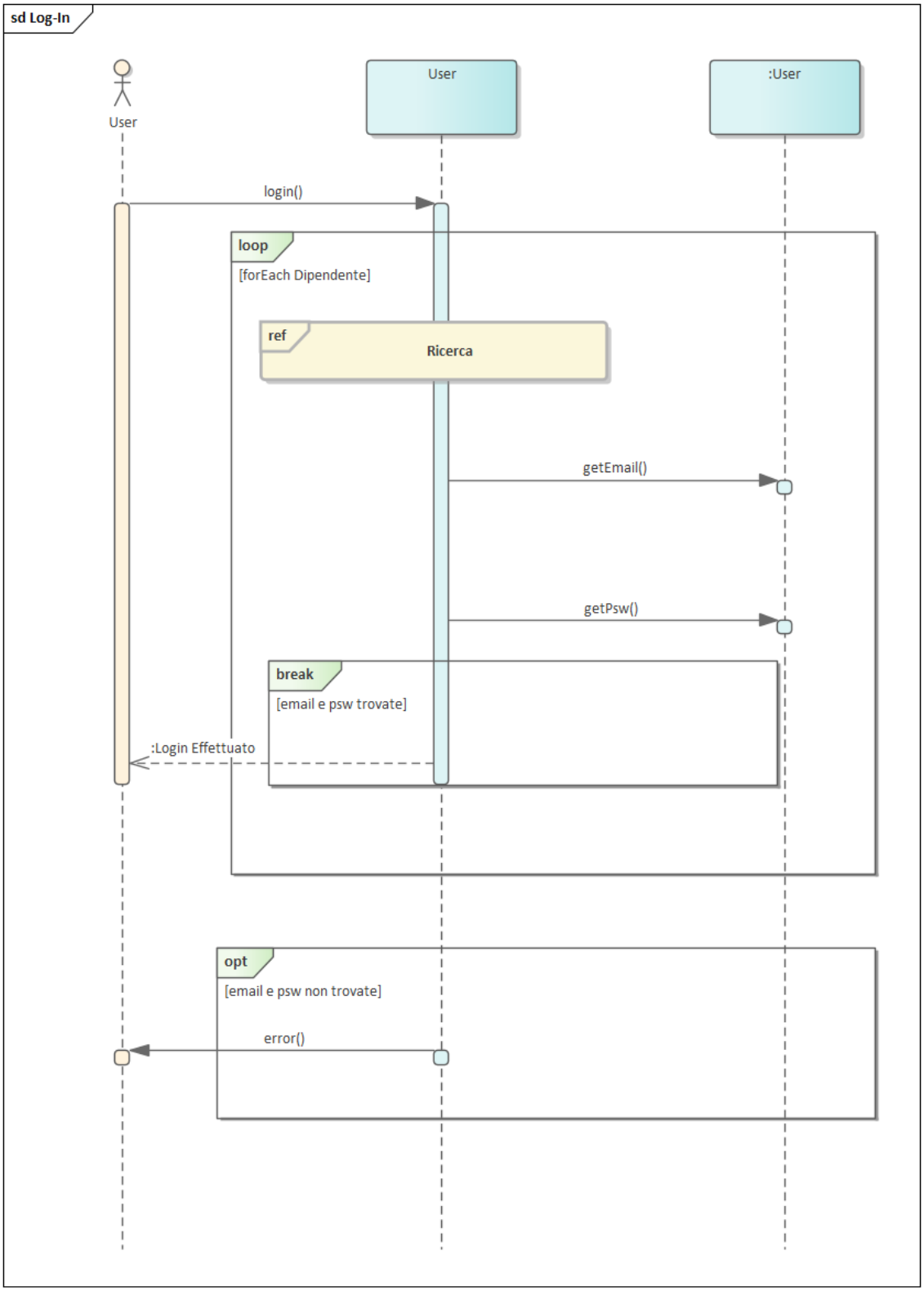


4.1.5 Vendita

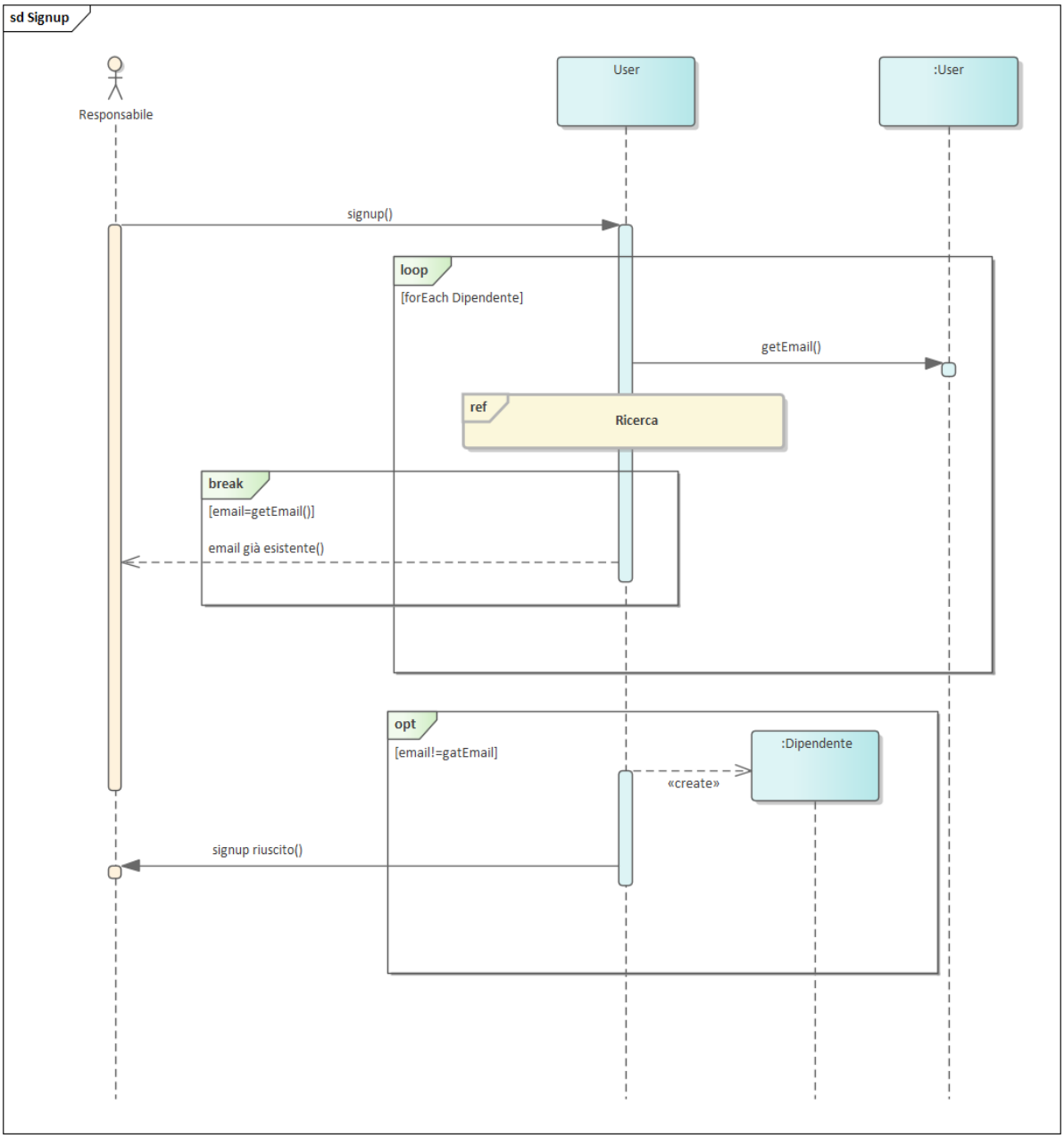

class Vendita
«enumeration»
MetodoPagamento
PAYPAL
CARTA
CONTANTI
User::User
- email: String
- id: int
- method: MetodoPagamento
- password_hash: str
- ruolo: RuoloDipendente
1
«makes»
0..*
Vendita
- data_vendita: datetime
- dipendente: Dipendente
- id: int
- metodo_pagamento
- oggetti: List[OggettoVendita]
1
«has»
1..*
Prodotto::VarianteProdotto
- brand: Brand
- dimensione: Dimensione
- id: int
- modello: Modello
- potenza: Potenza
- statusProdotto: StatusProdotto
1
«refers to»
0..*
OggettoVendita
- id_prodotto: int
- quantita: int

4.2 Diagrammi di sequenza

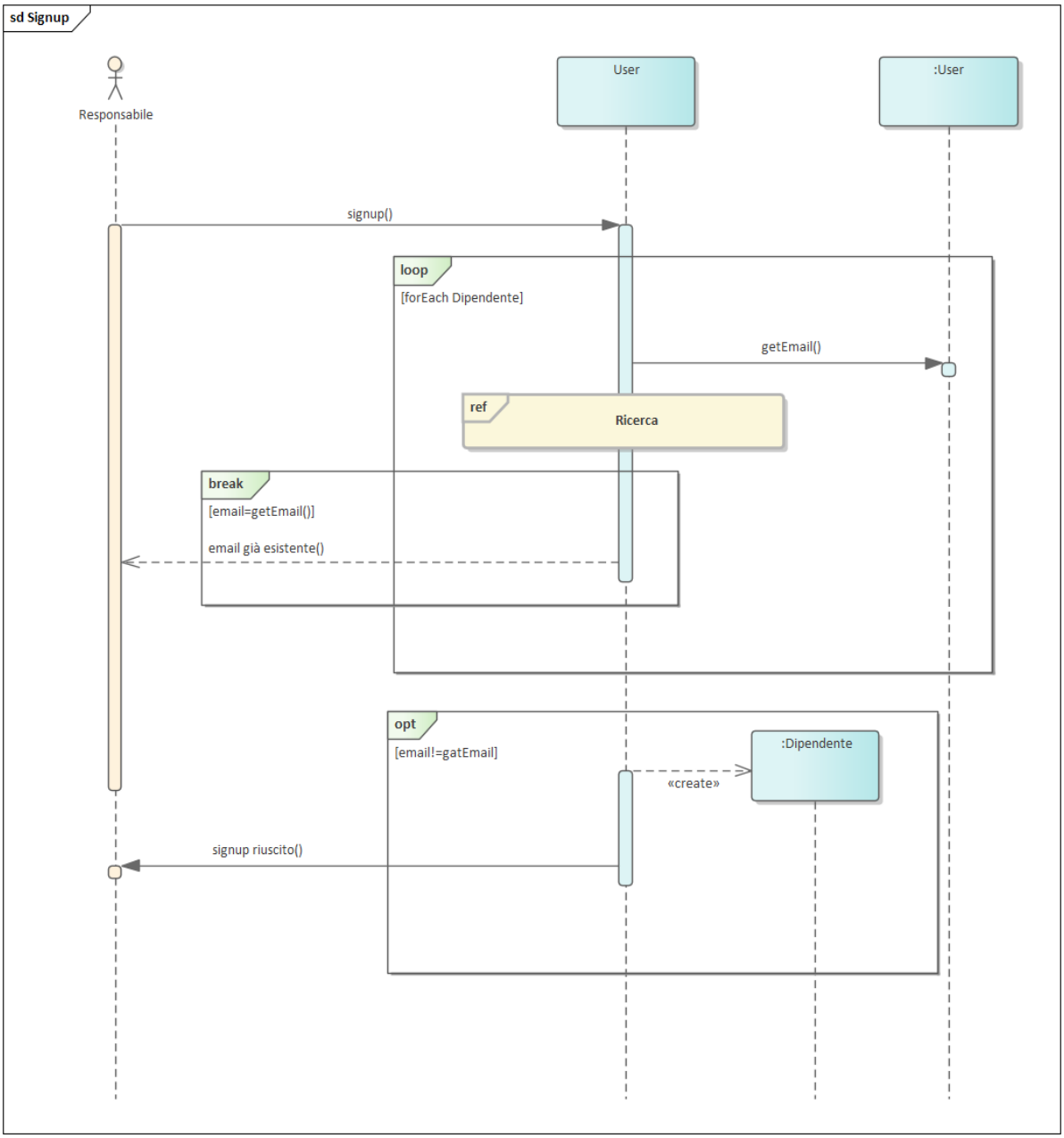
4.2.1 Login



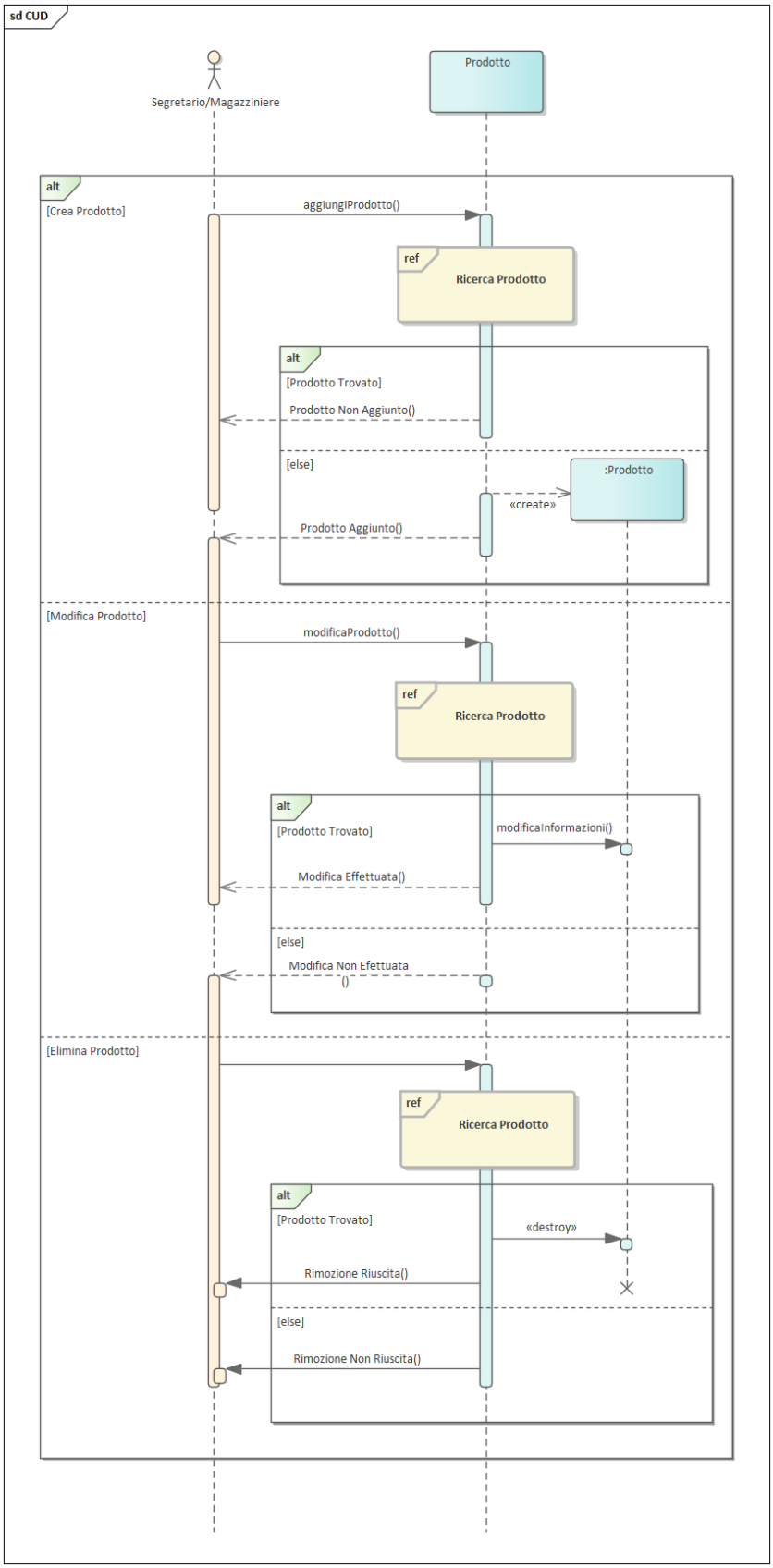
4.2.2 Signup


sd Signup
Responsabile
User
:User
signup()
loop
[forEach Dipendente]
getEmail()
ref
Ricerca
break
[email=getEmail()]
email già esistente()
opt
[email!=gatEmail]
:Dipendente
«create»
signup riuscito()

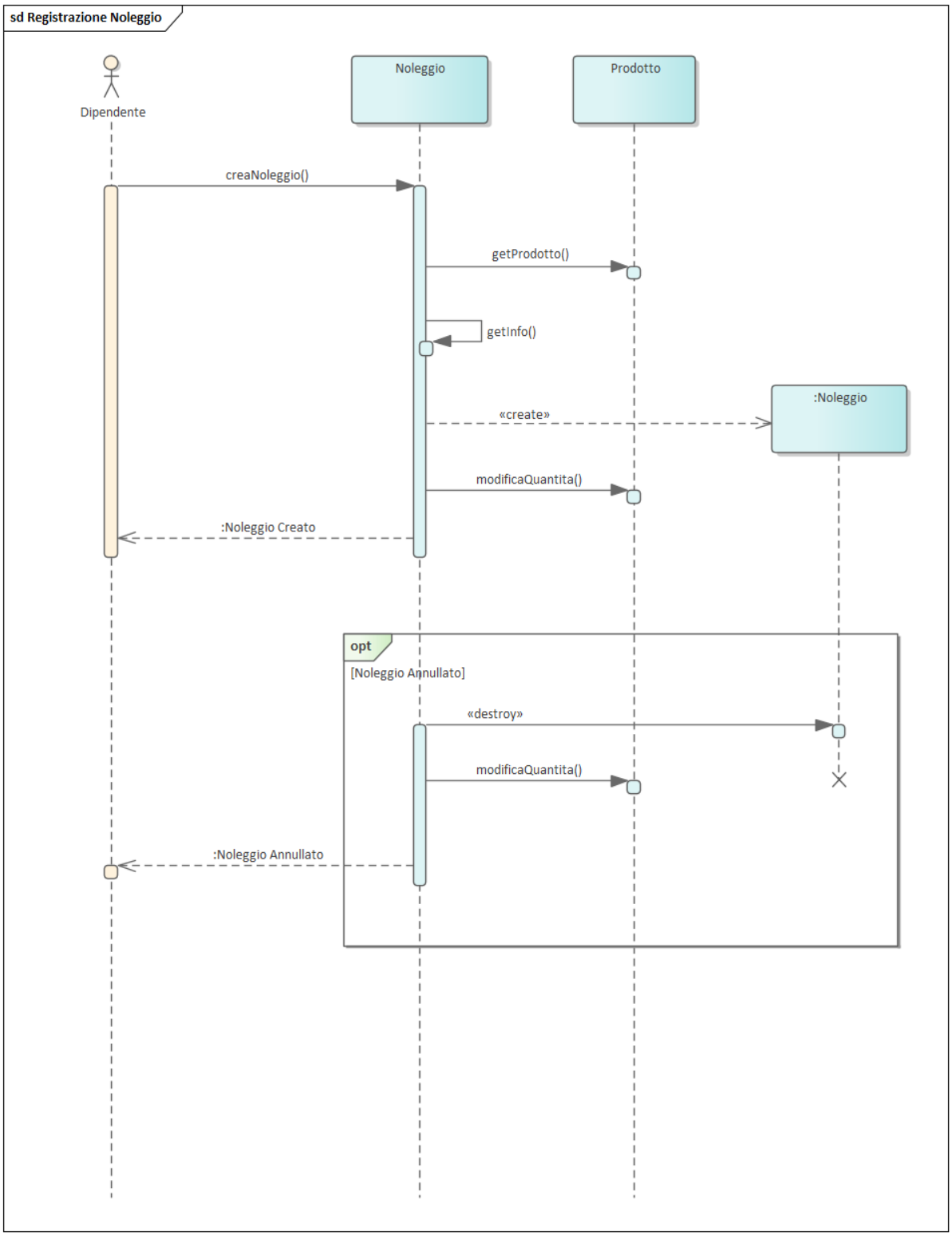
4.2.3 Customer Singup



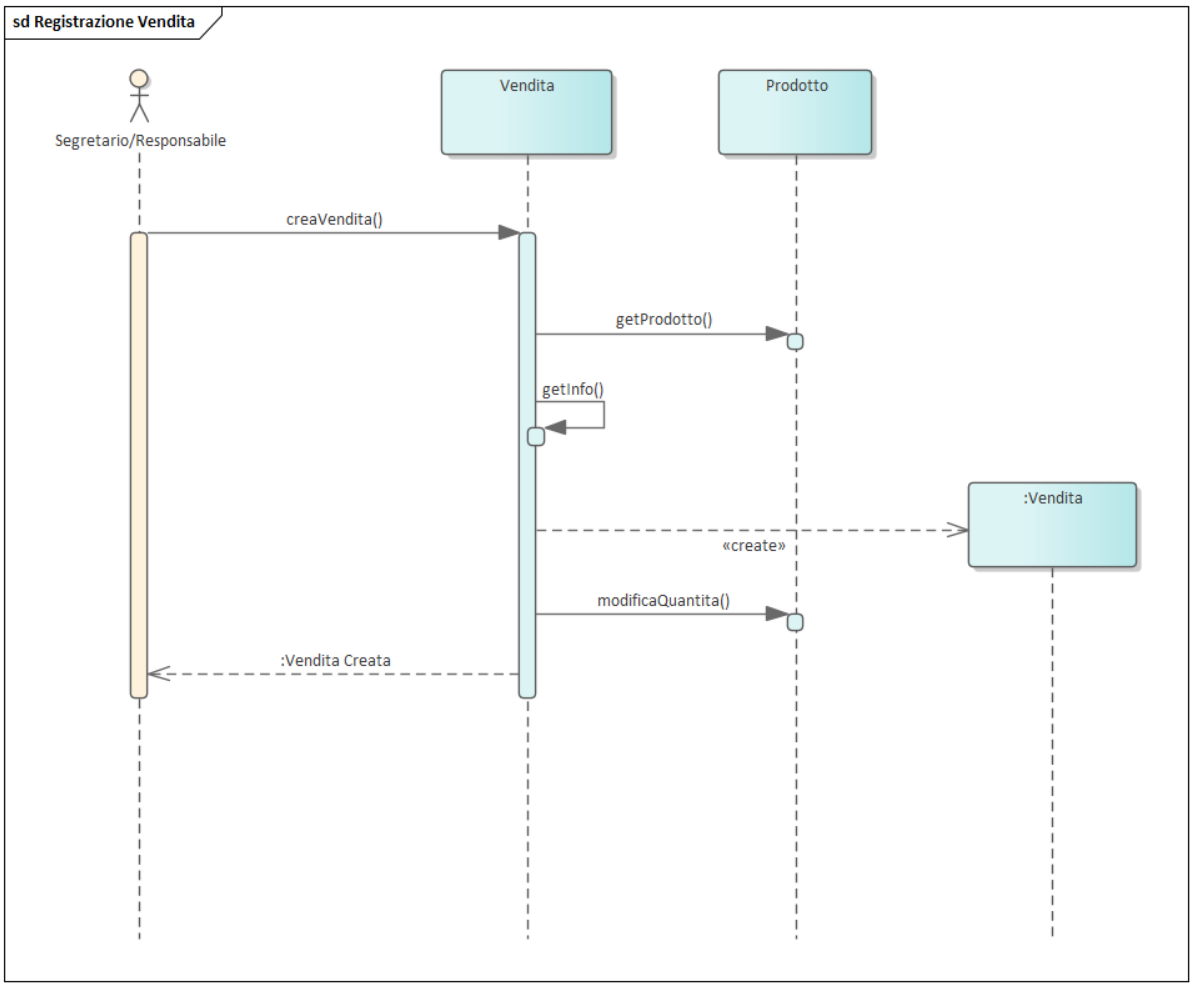
4.2.4 CUD


sd CUD
Segretario/Magazziniere
Prodotto
alt
[Crea Prodotto]
aggiungiProdotto()
ref
Ricerca Prodotto
alt
[Prodotto Trovato]
Prodotto Non Aggiunto()
[else]
:Prodotto
«create»
Prodotto Aggiunto()
[Modifica Prodotto]
modificaProdotto()
ref
Ricerca Prodotto
alt
[Prodotto Trovato]
modificaInformazioni()
Modifica Effettuata()
[else]
Modifica Non Efettuata
()
[Elimina Prodotto]
ref
Ricerca Prodotto
alt
[Prodotto Trovato]
«destroy»
Rimozione Riuscita()
[else]
Rimozione Non Riuscita()

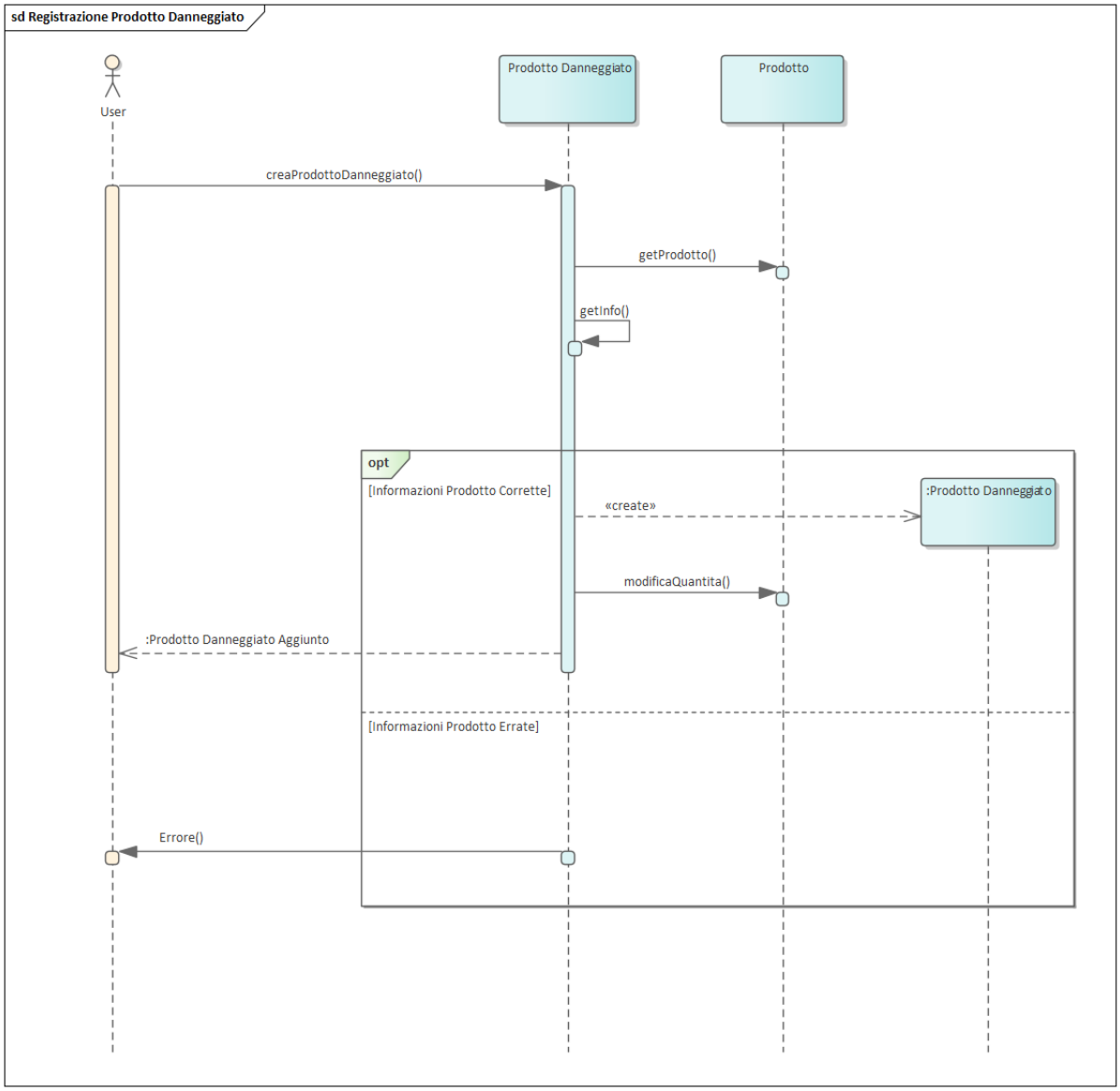
4.2.5 Registrazione Noleggio



4.2.6 Registrazione Vendita

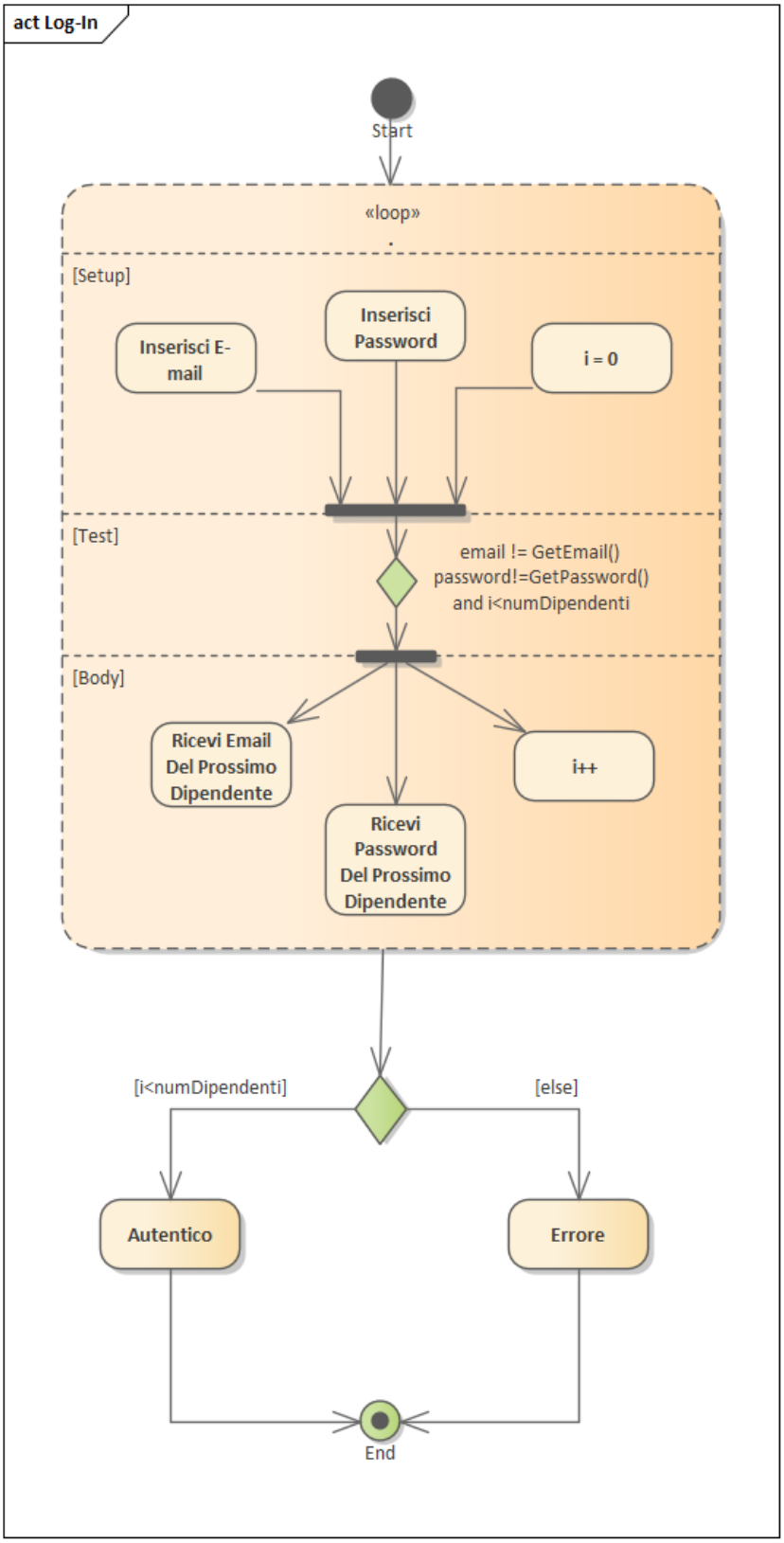


4.2.7 Registrazione Prodotto Danneggiato

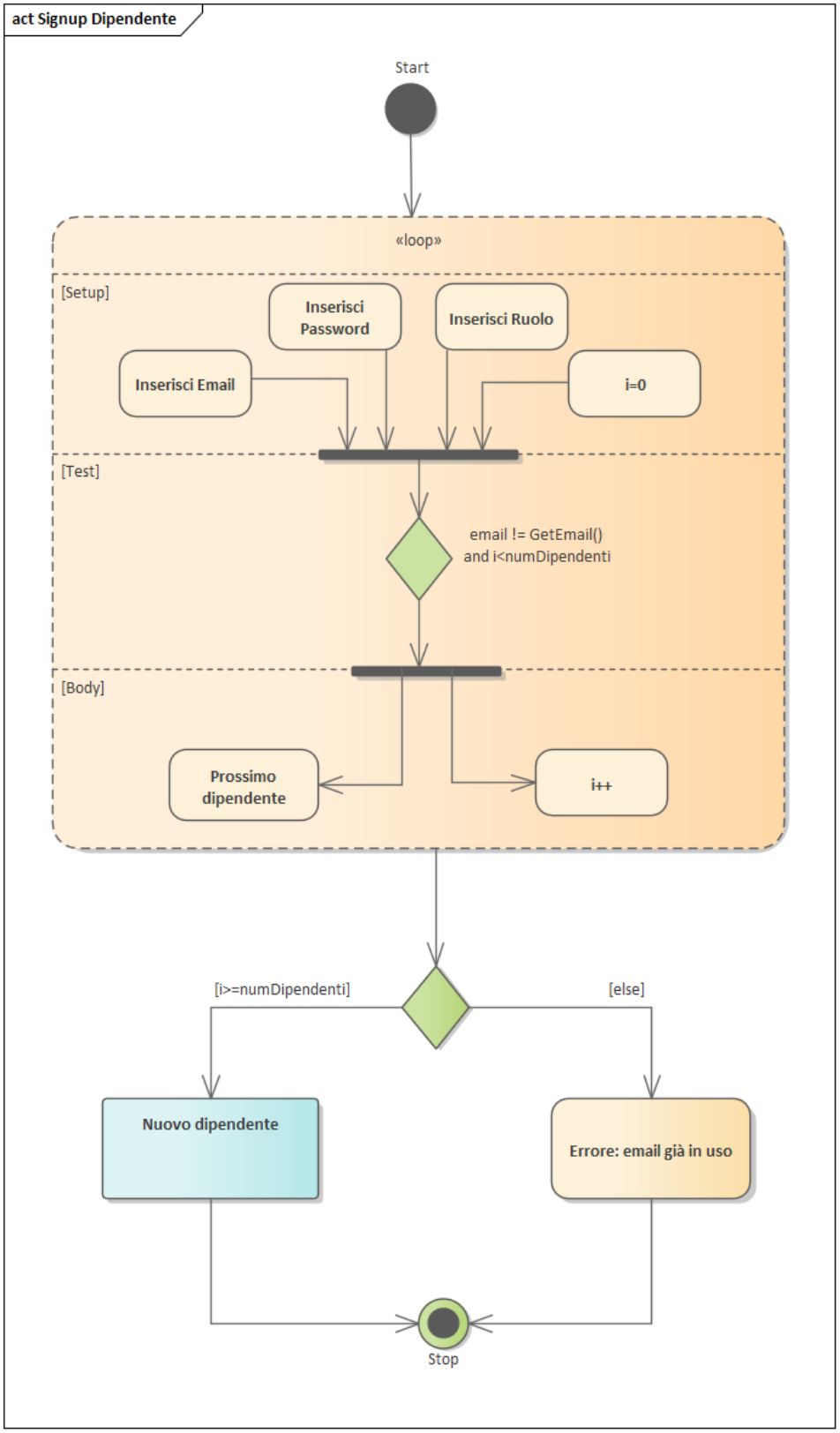


4.3 Diagrammi di attività

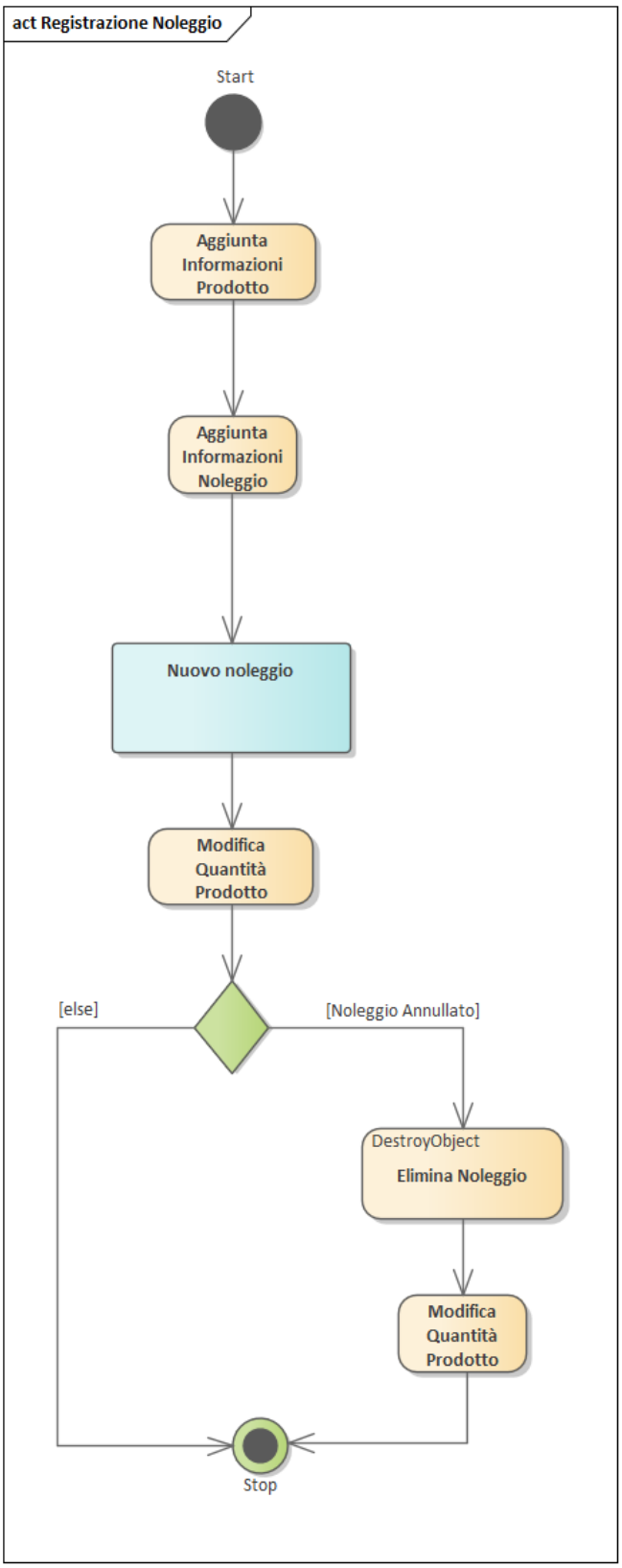
4.3.1 Login



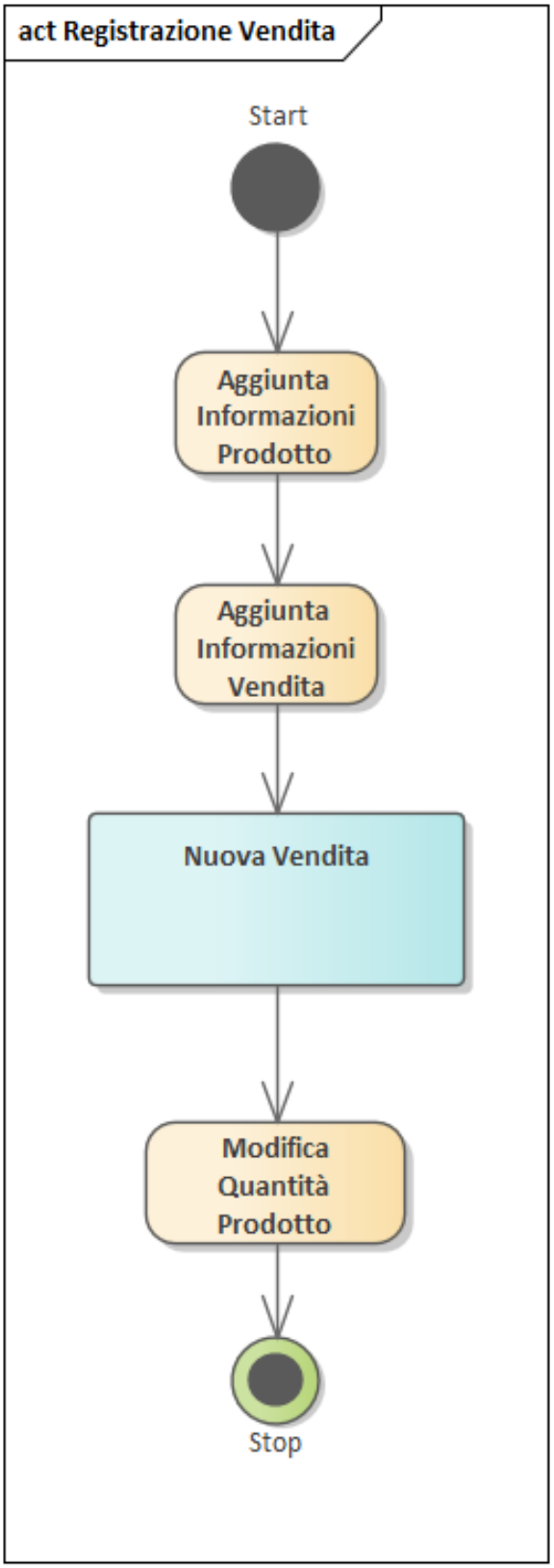
4.3.2 Signup


act Signup Dipendente
Start
«loop»
[Setup]
Inserisci Password
Inserisci Ruolo
Inserisci Email
i=0
[Test]
email != GetEmail()
and i<numDipendenti
[Body]
Prossimo dipendente
i++
[i>=numDipendenti]
[else]
Nuovo dipendente
Errore: email già in uso
Stop

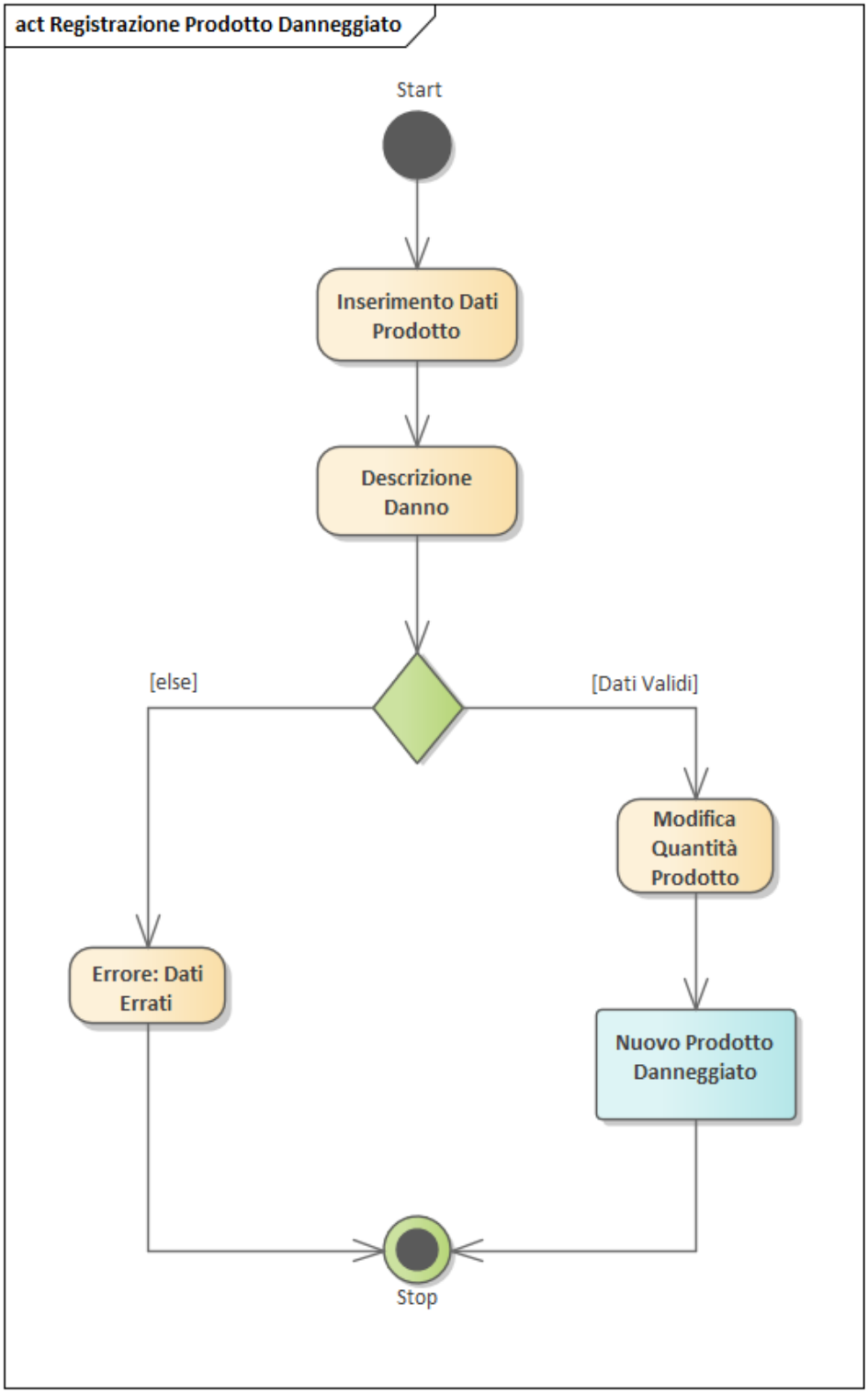
4.3.3 Registrazione Noleggio



4.3.4 Registrazione Vendita



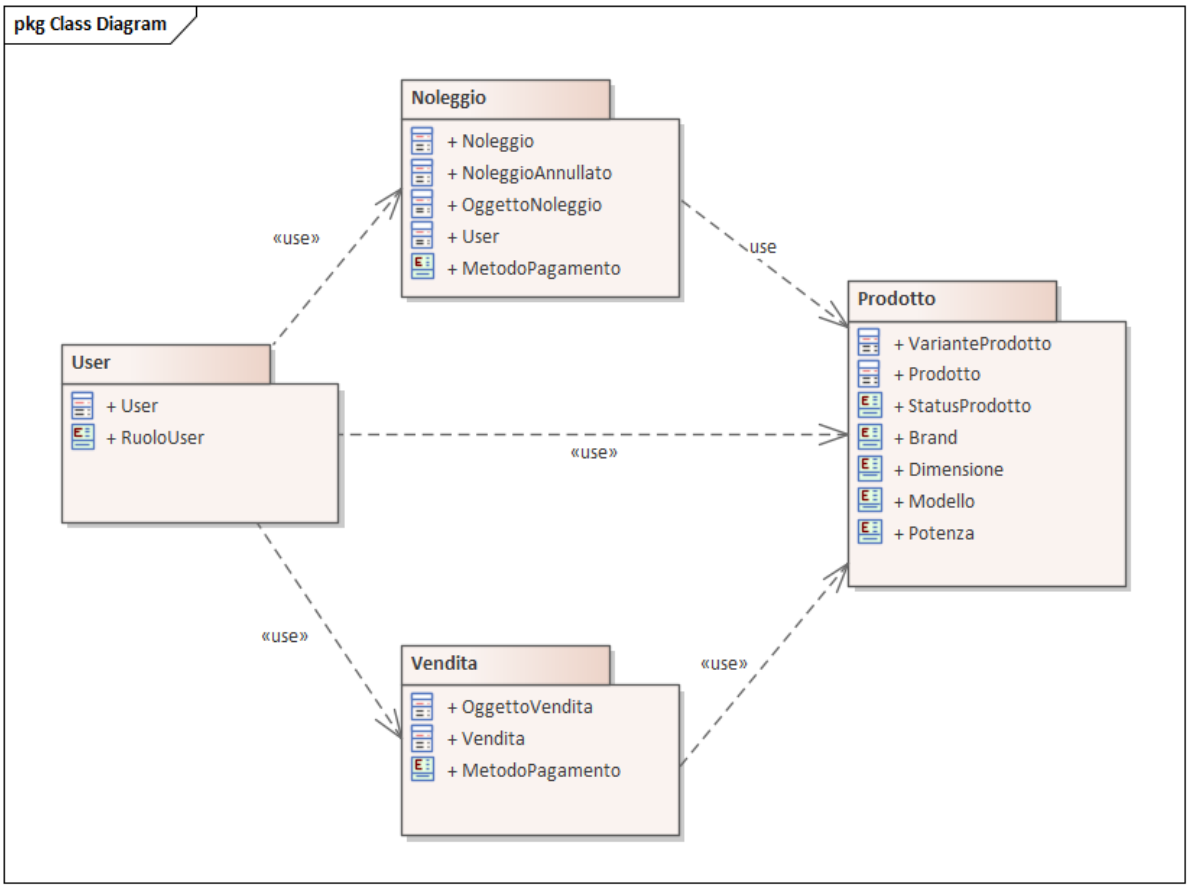
4.3.5 Registrazione Prodotto Danneggiato



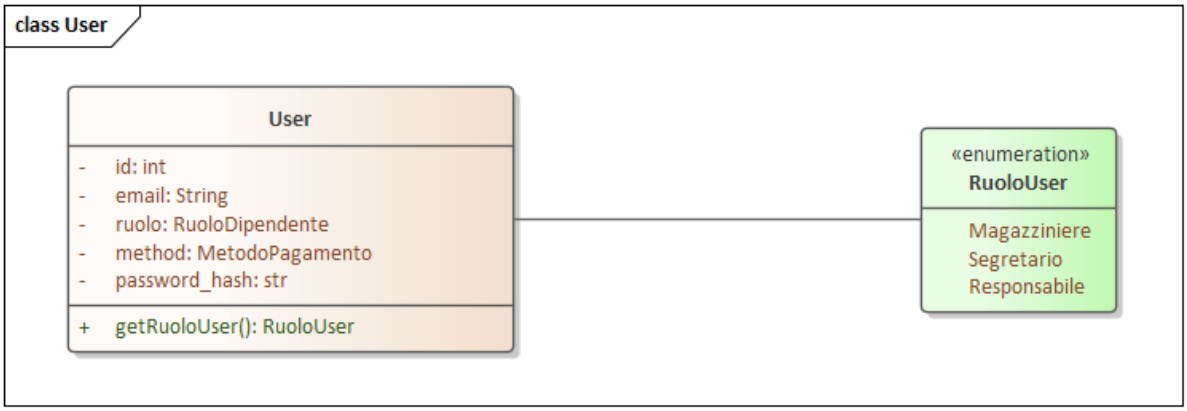
5 Progettazione

5.1 Classi di progettazione

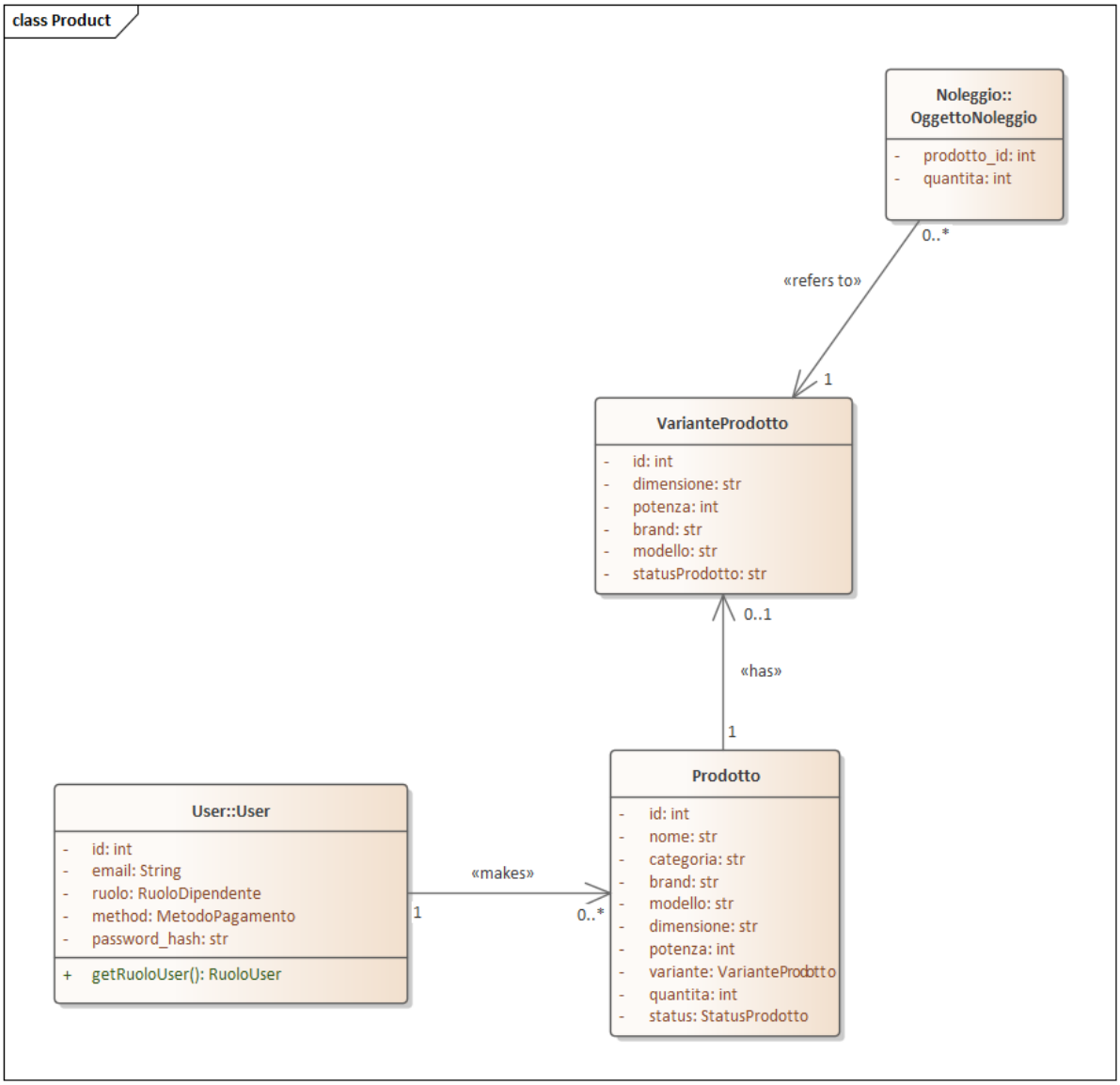
5.1.1 Classi di progettazione: Package



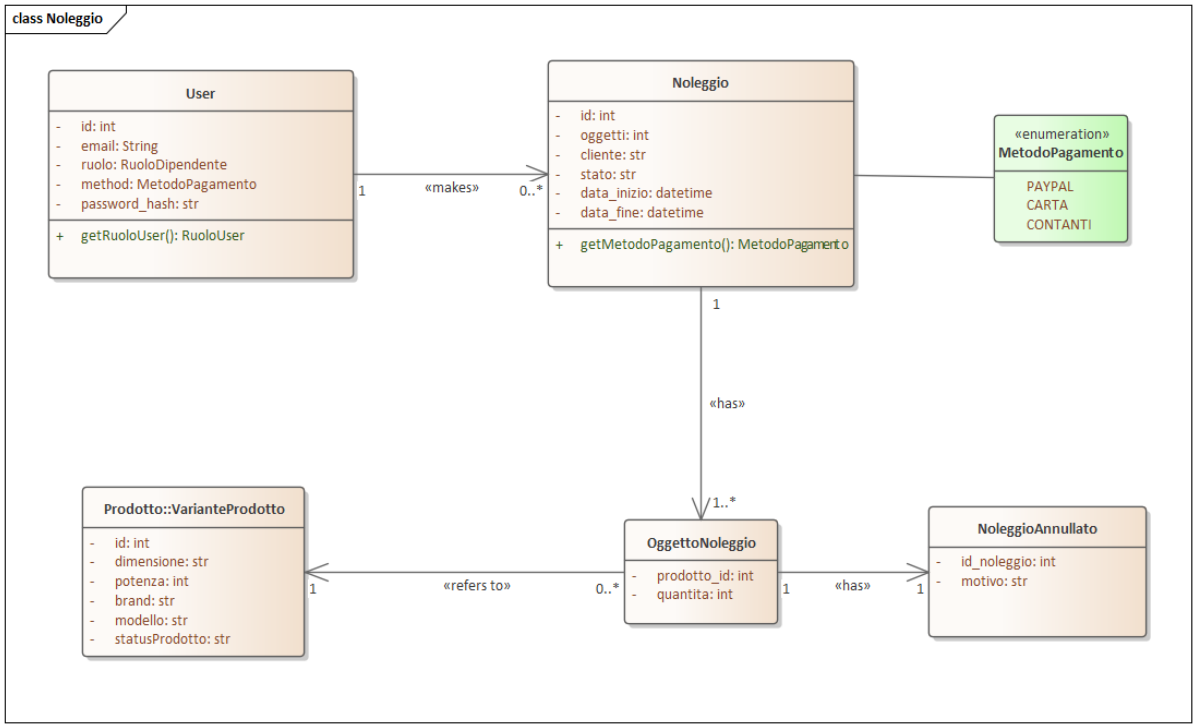
5.1.2 Classi di progettazione: User



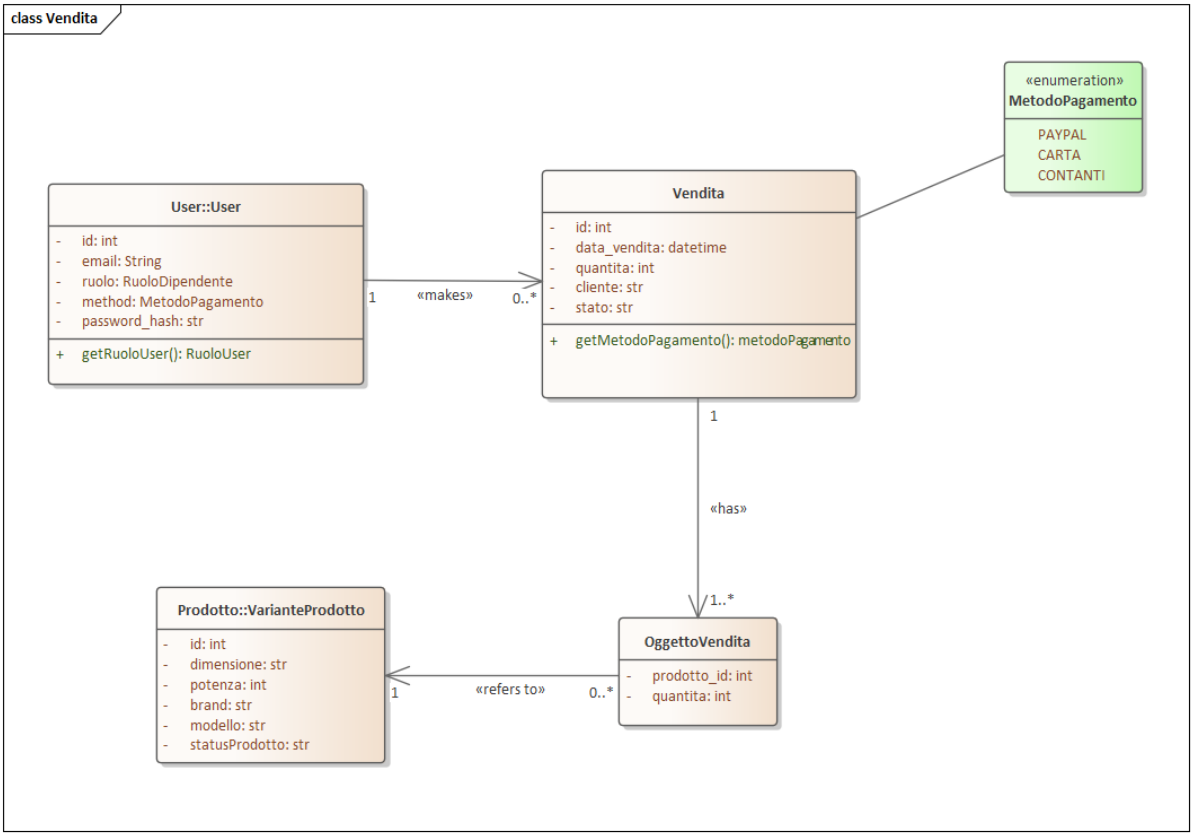
5.1.3 Classi di progettazione: Prodotto



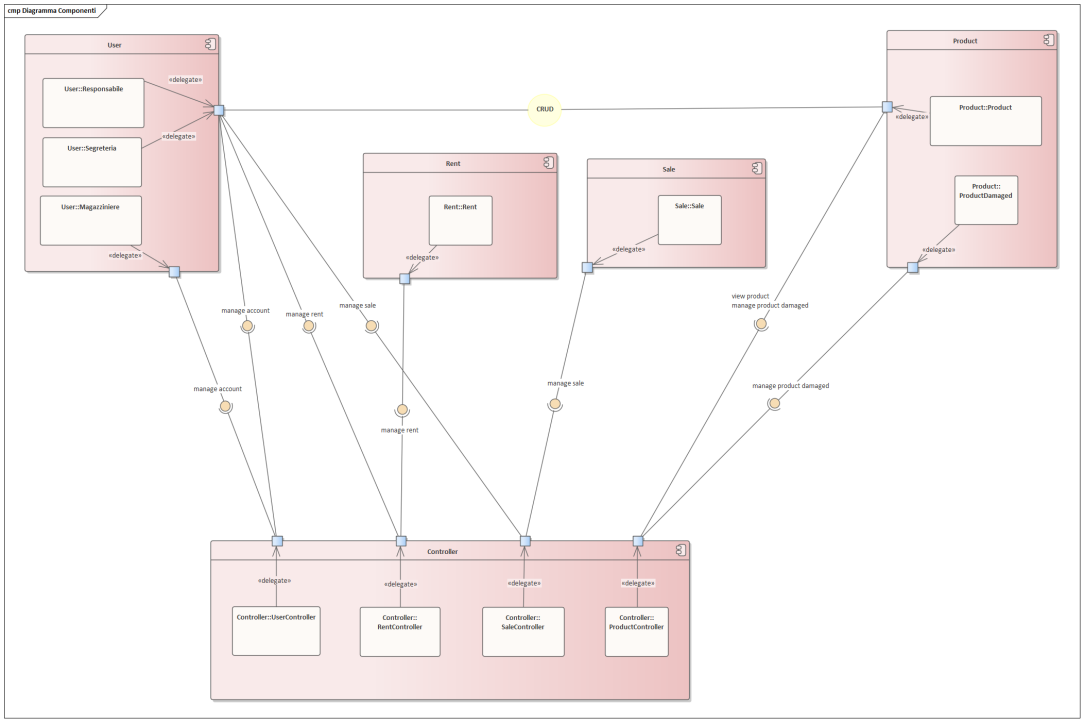
5.1.4 Classi di progettazione: Noleggio



5.1.5 Classi di progettazione: Vendita

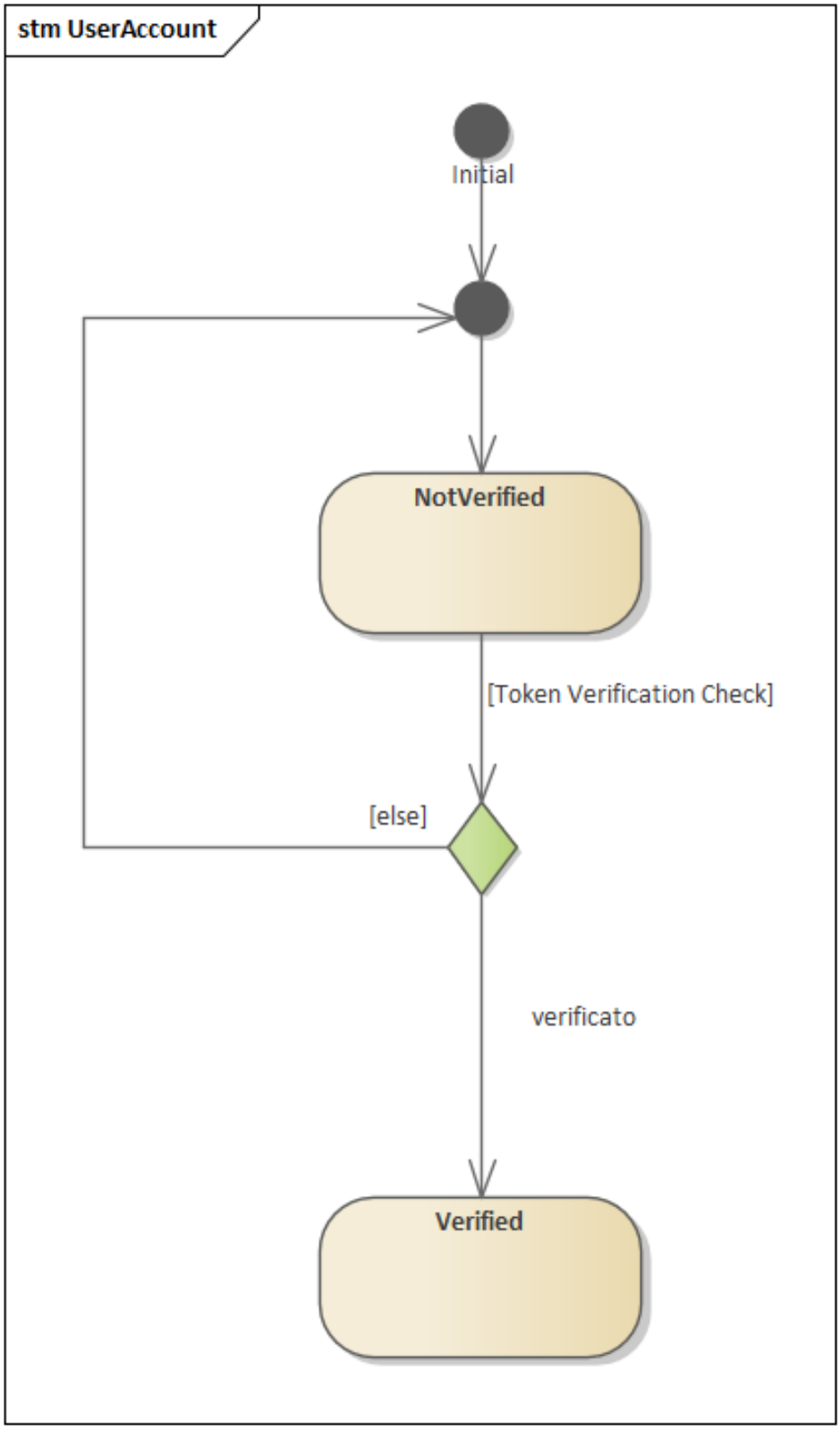


5.2 Diagramma dei componenti

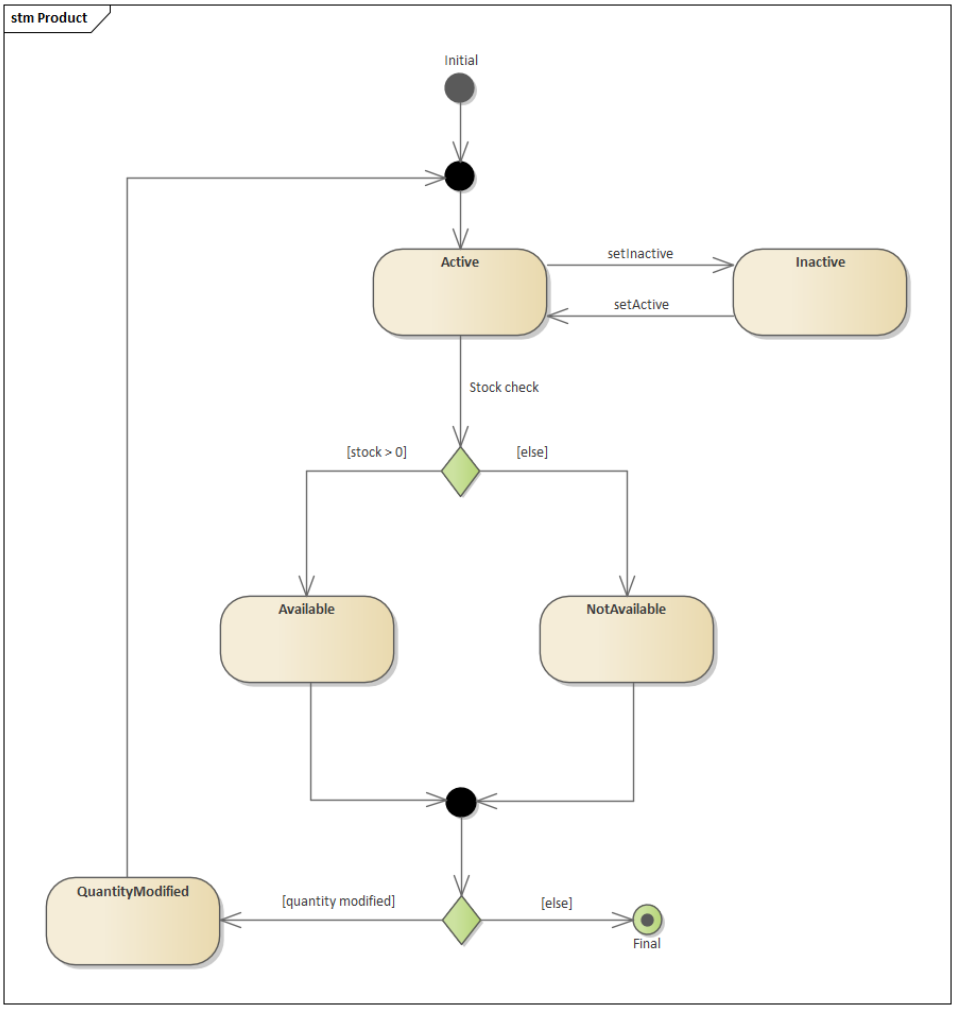


5.3 Diagrammi delle macchine a stati

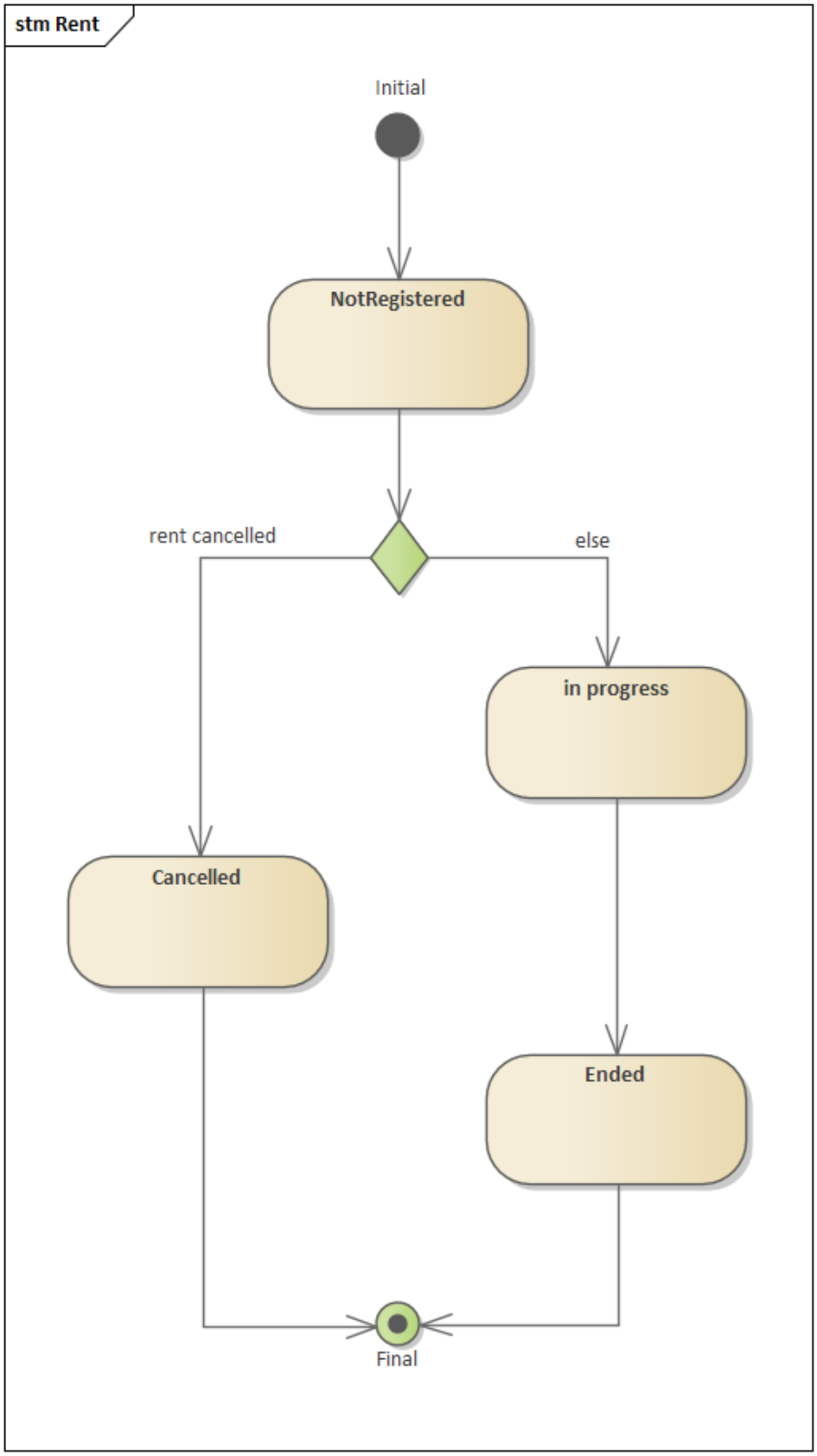
5.3.1 User



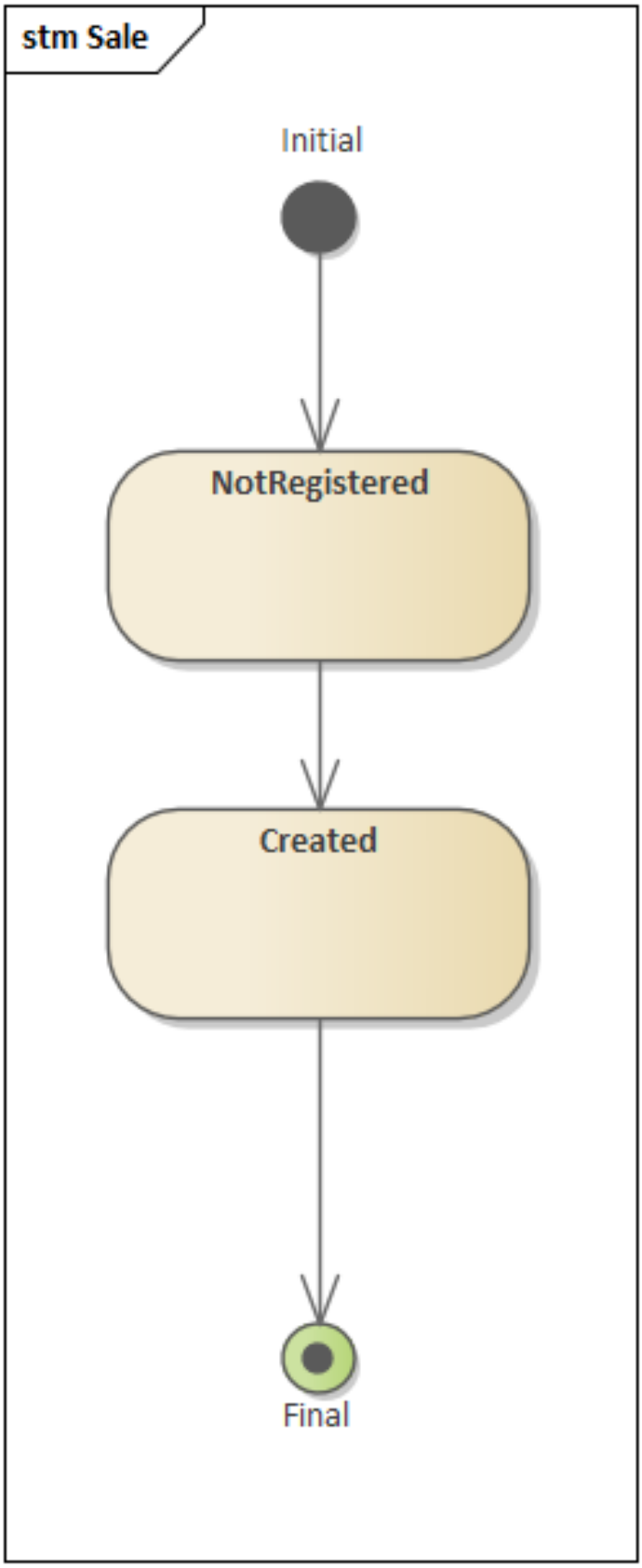
5.3.2 Prodotto



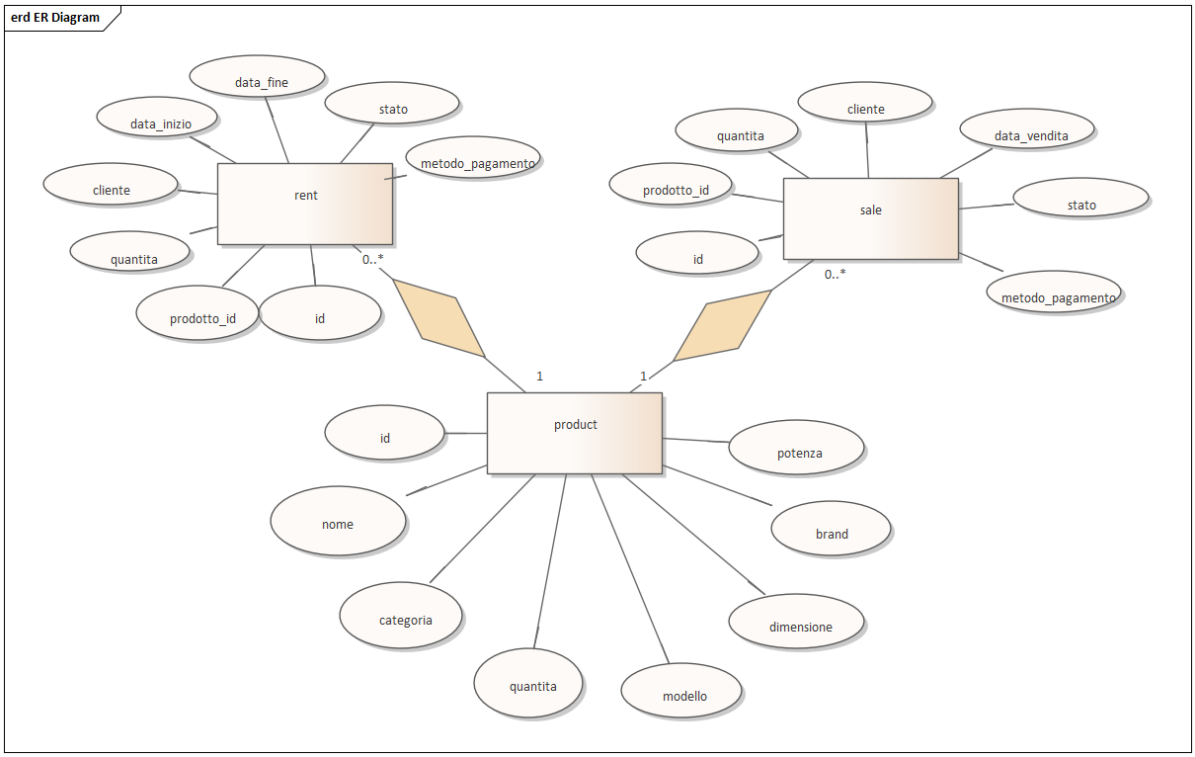
5.3.3 Noleggio



5.3.4 Vendita

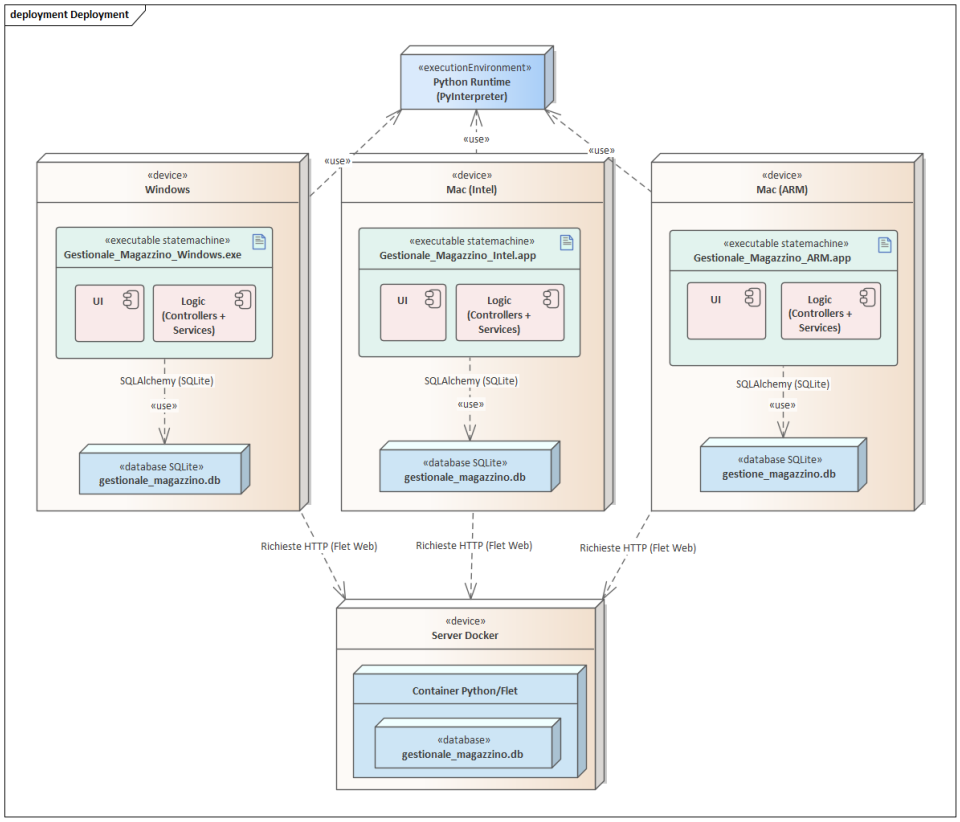


5.4 Diagramma entità-relazioni



6 Implementazione

6.1 Diagrammi di deployment



6.2 Tecnologie utilizzate

6.2.1 Tecnologie Front-End

Flet
Flet è un framework open-source scritto in Python che permette di creare interfacce grafiche moderne in modo semplice e veloce, mantenendo un approccio dichiarativo simile a Flutter. Nel nostro progetto è stato scelto per la sua facilità di integrazione con Python e per la rapidità nello sviluppo dell'interfaccia utente. Durante la realizzazione dell'applicazione abbiamo fatto ampio uso della documentazione ufficiale di Flet disponibile su https://flet.dev/docs/controls, in particolare per consultare e sperimentare i numerosi controlli grafici disponibili, così da ottenere un'interfaccia coerente e professionale.

6.2.2 Tecnologie Back-End

Python 3.11.9 e Virtual Environment (venv) Il back-end del progetto è stato sviluppato utilizzando il linguaggio Python nella versione 3.11.9, per garantire la compatibilità con tutte le librerie utilizzate, in particolare pydantic, SQLAlchemy e bcrypt. È stato predisposto un ambiente virtuale (venv) creato e testato specificatamente con questa versione di Python, così da assicurare la riproducibilità dell'ambiente di sviluppo ed evitare conflitti con versioni diverse. Per avviare il progetto, è necessario creare e attivare il venv con Python 3.11.9 e installare i requisiti presenti nel file `requirements.txt`.

SQLite
SQLite è un sistema di gestione di database relazionali leggero e incorporato, ideale per applicazioni desktop. Non richiede un server esterno e memorizza i dati in un singolo file locale, semplificando la distribuzione e la gestione dei dati. È integrato nativamente in Python tramite il modulo sqlite3.

SQLAlchemy
SQLAlchemy è un Object Relational Mapper (ORM) per Python che consente di interagire con il database relazionale attraverso oggetti Python, evitando di scrivere direttamente query SQL complesse. È stato utilizzato per gestire in modo sicuro e organizzato tutte le operazioni di lettura e scrittura sui dati.

Pydantic
Pydantic è una libreria per la validazione dei dati basata su Python. Nel progetto è stata impiegata per definire gli schemi dei dati (ad esempio utenti, prodotti, noleggi, vendite e notifiche), garantendo che i valori rispettassero i tipi previsti e migliorando così la robustezza del sistema.

Passlib e Bcrypt
Per garantire la sicurezza delle password degli utenti, è stato utilizzato il modulo Passlib con l'algoritmo Bcrypt. Questo ha consentito di memorizzare le password in forma criptata, rendendo più sicuro il processo di autenticazione.

6.2.3 Tecnologie Cloud

Docker
Docker è una piattaforma open source che consente di eseguire applicazioni all'interno di container isolati. È stato utilizzato per distribuire il progetto anche in modalità web, permettendo di eseguirlo da qualsiasi browser senza necessità di installazioni locali complesse.

Docker Compose
Docker Compose è stato utilizzato per gestire e orchestrare l'avvio del container Docker dell'applicazione in modo semplice e rapido.

6.2.4 Tecnologie di versionamento

Git
Git è stato utilizzato per tenere traccia delle modifiche del codice sorgente e per garantire un controllo puntuale delle versioni durante lo sviluppo del progetto.
Github
GitHub è stato utilizzato come piattaforma di hosting per il repository Git del progetto. Ha consentito la sincronizzazione continua del codice, il backup centralizzato e la collaborazione di più sviluppatori.

6.2.5 Altre tecnologie utilizzate

Enterprise Architect
Enterprise Architect è un software professionale per la modellazione UML utilizzato per progettare l'architettura del sistema prima dello sviluppo. È stato impiegato per realizzare i diagrammi UML richiesti dal progetto.

6.3 Mockup

Figure 1: Login

Figure 2: Recupera Password

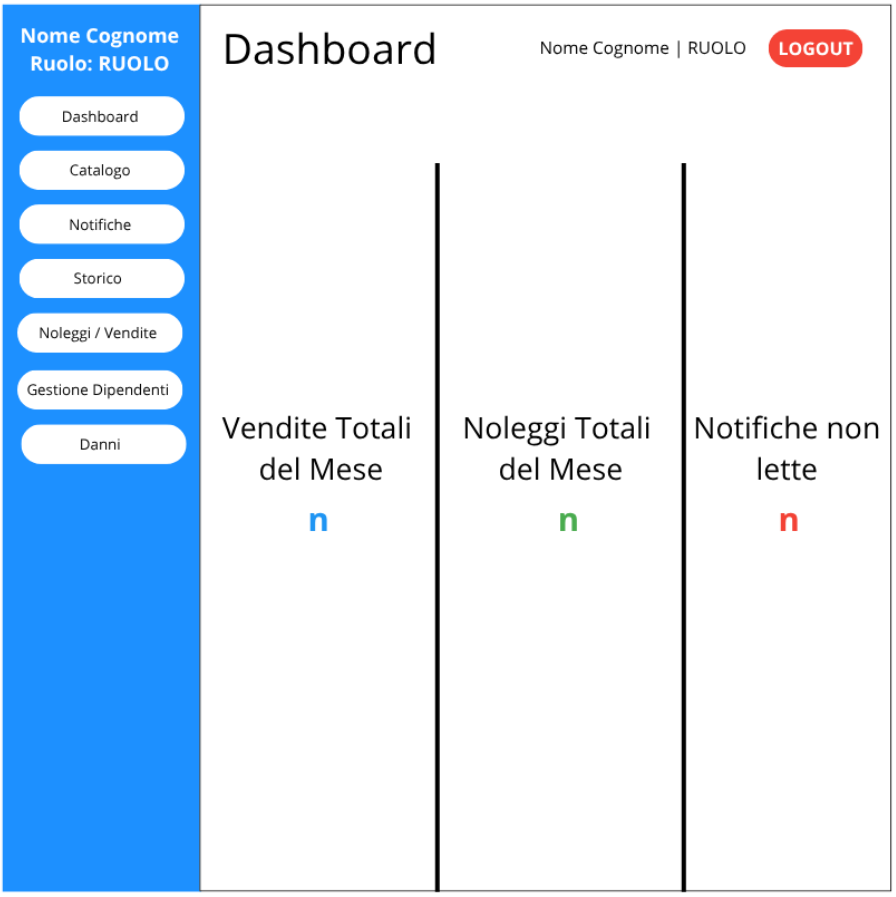


Figure 3: Dashboard

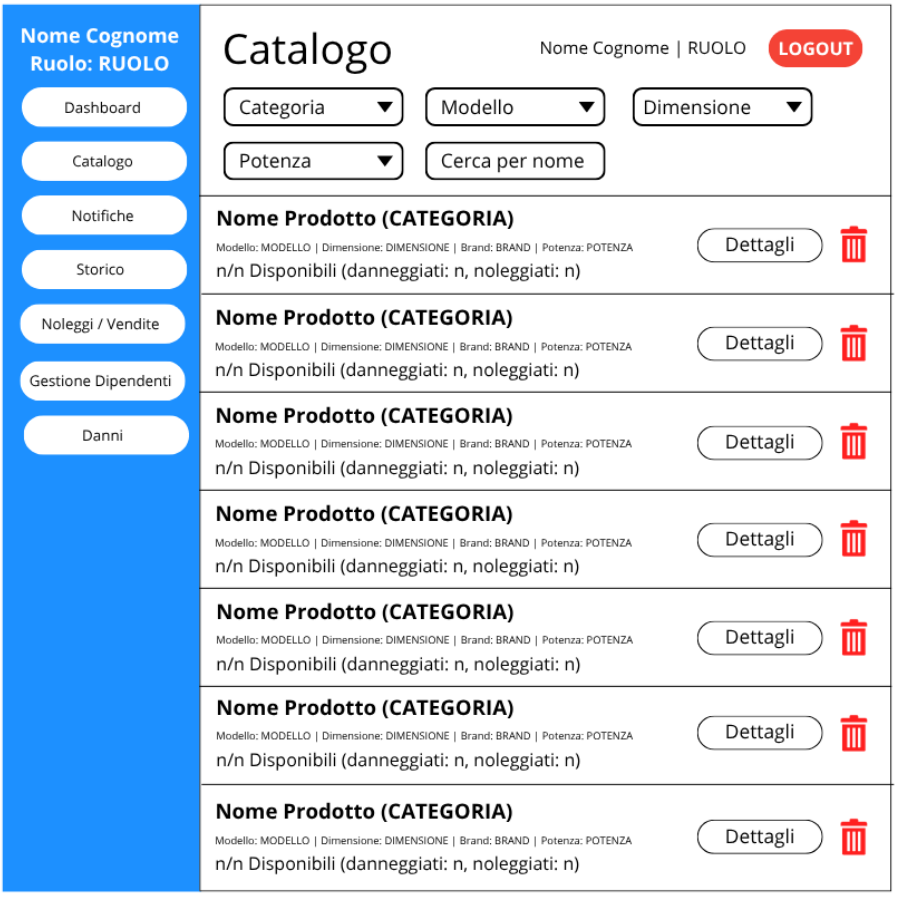


Figure 4: Catalogo

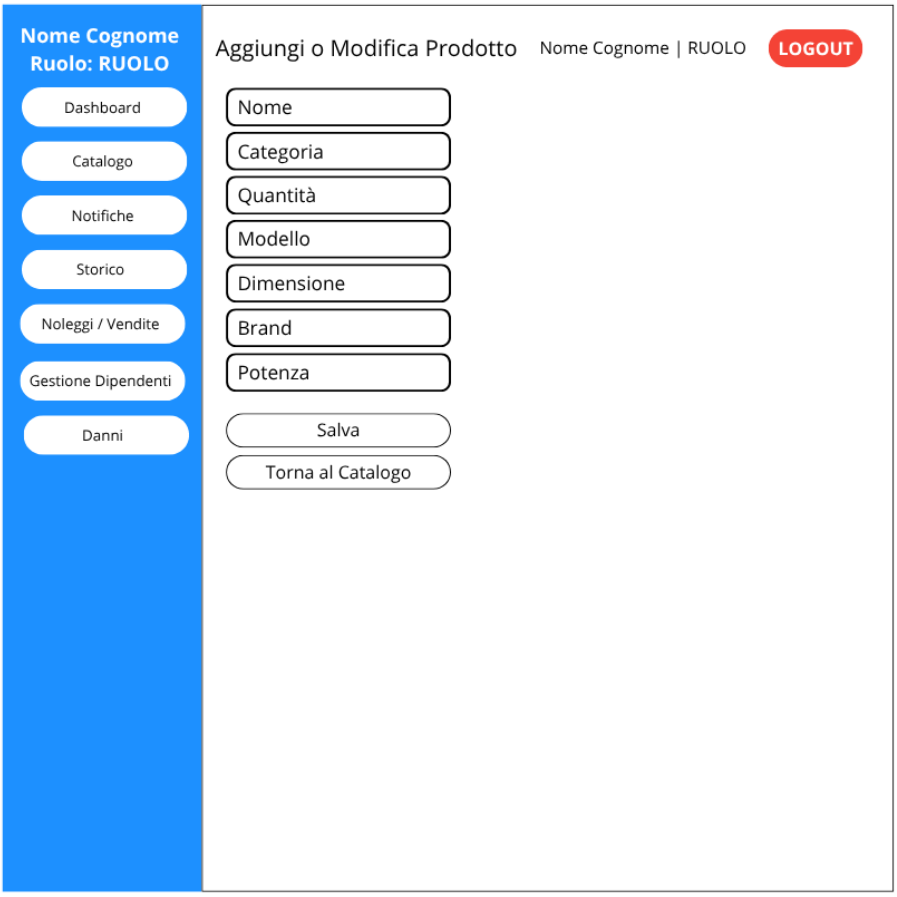


Figure 5: Aggiungi o Modifica Prodotto

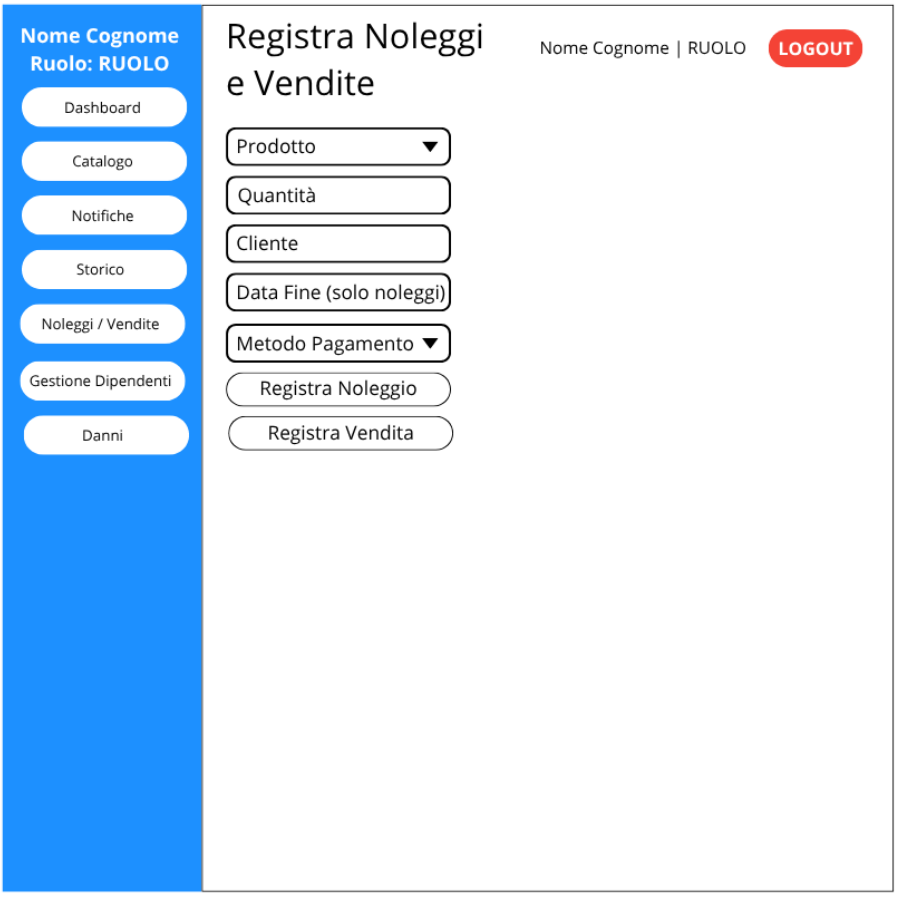


Figure 6: Registra Noleggi e Vendite

Figure 7: Modifica Noleggio

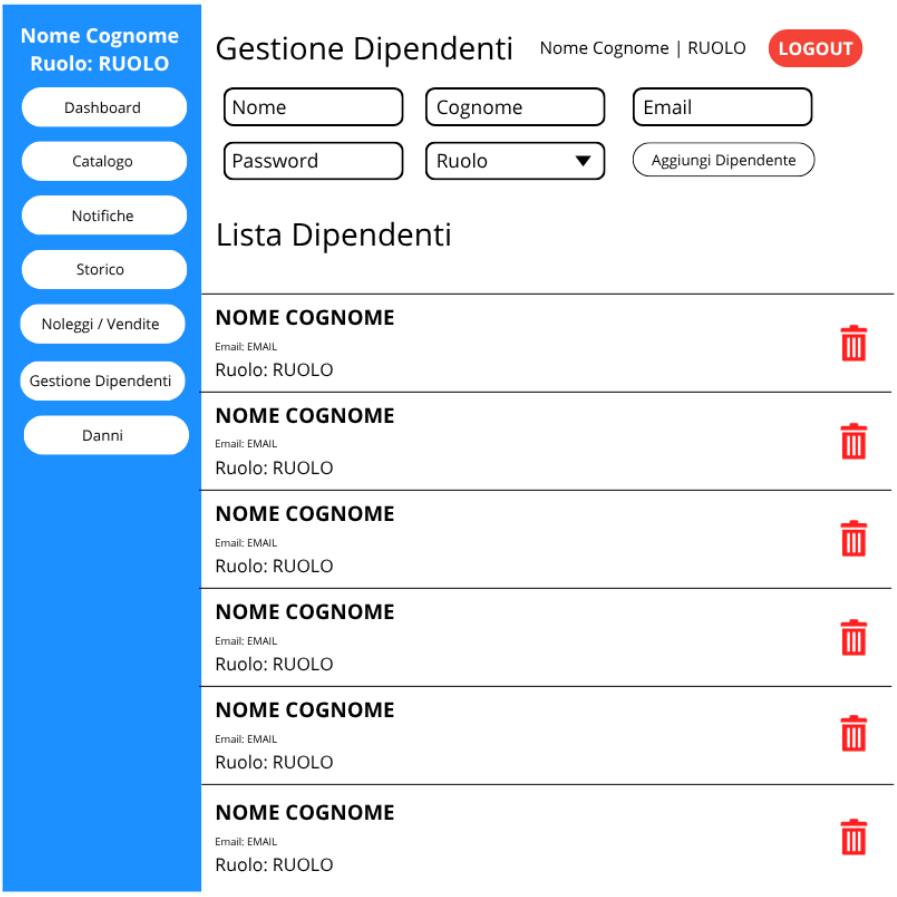


Figure 8: Gestione Dipendenti

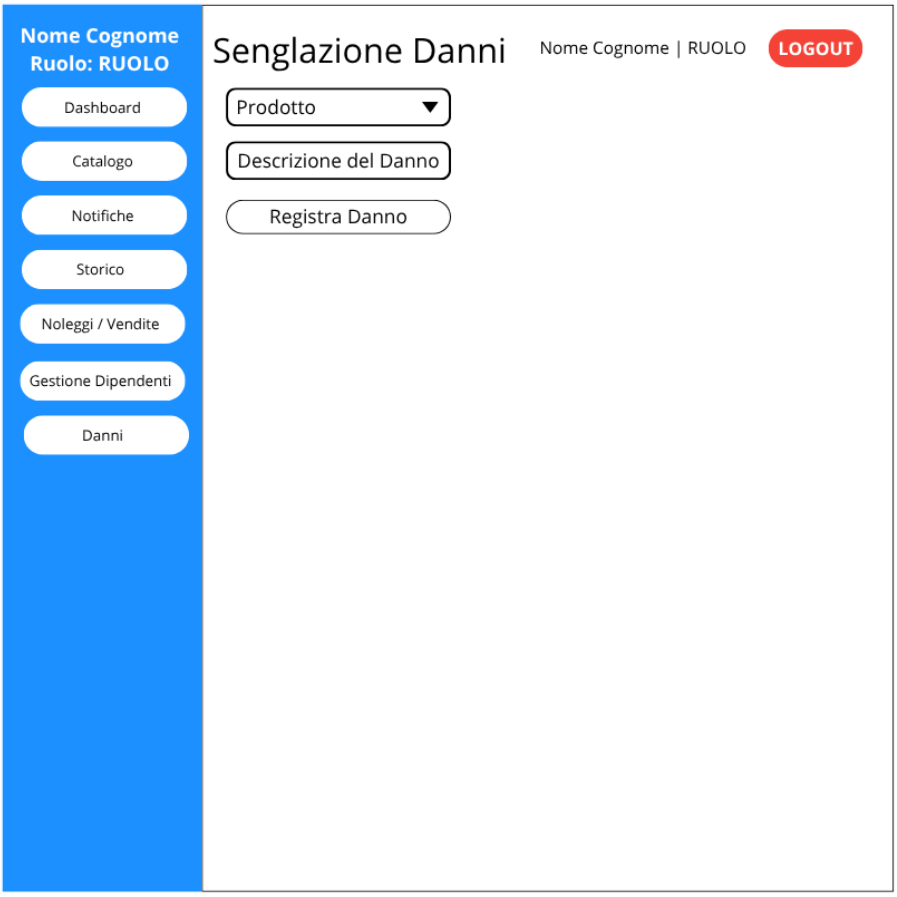


Figure 9: Segnalazione Danni

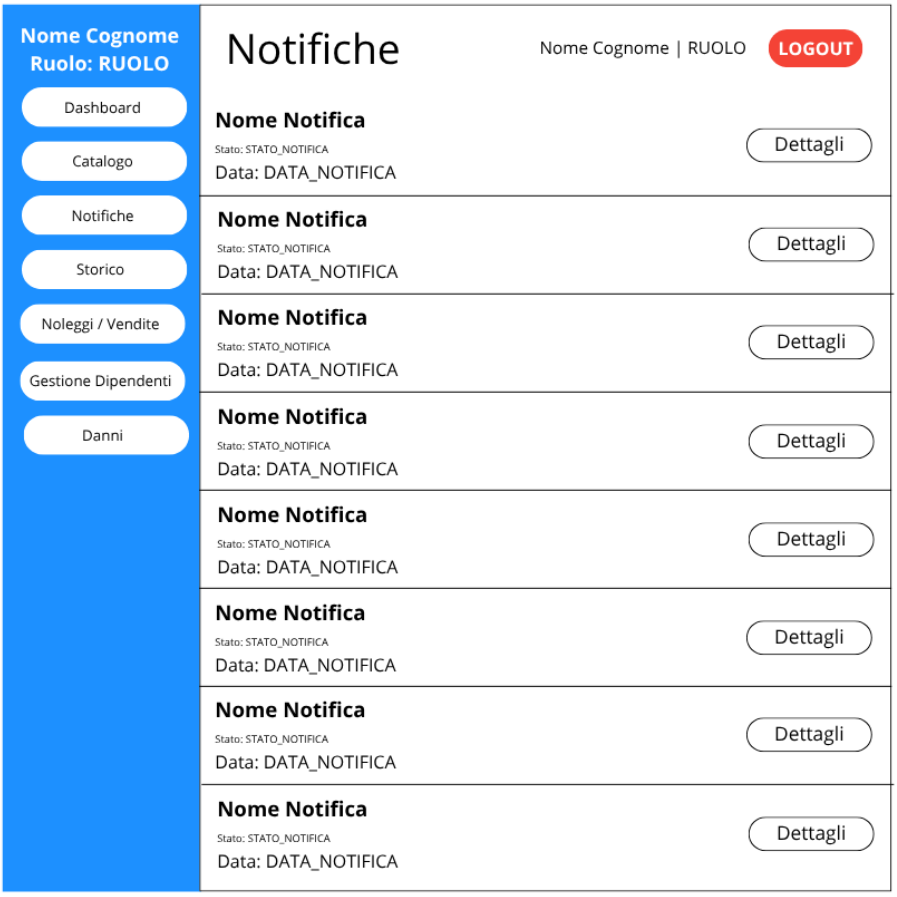


Figure 10: Notifiche

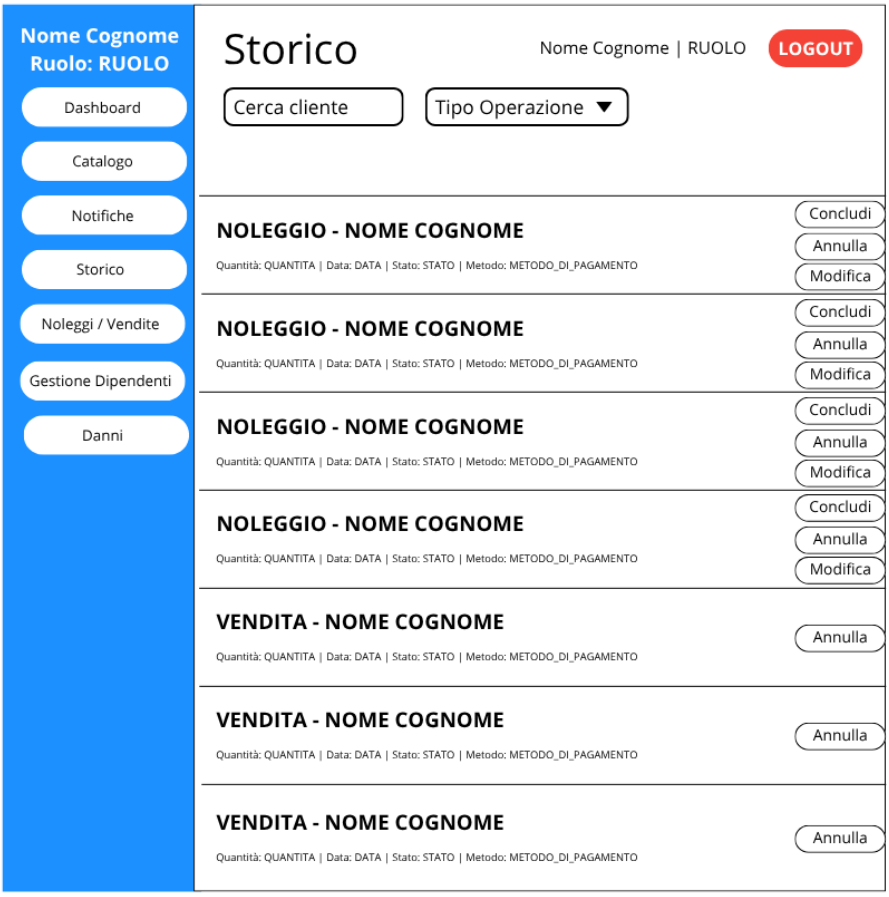


Figure 11: Storico

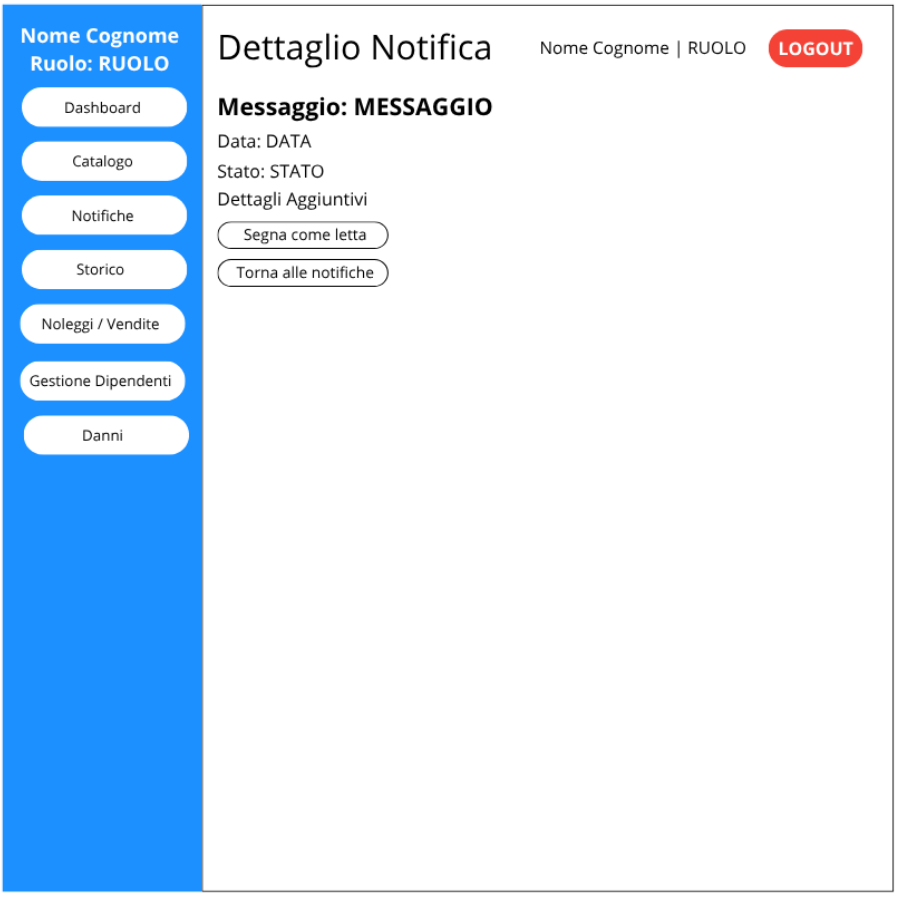


Figure 12: Dettaglio Notifica

6.4 Descrizione unit tests

Sono stati sviluppati diversi unit test per verificare il corretto funzionamento dei servizi principali del gestionale.

Test Auth Service

Questo test verifica la correttezza del sistema di autenticazione e registrazione degli utenti. Controlla che la registrazione accetti solo dati validi, rifiuti email duplicate e permetta l'accesso solo con credenziali corrette. In caso di password errata l'accesso viene giustamente negato.

```python
import unittest
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from app.models.database import Base
from app.services.auth_service import register_user, authenticate_user
from app.schemas.user_schema import UserCreate, UserRole
from sqlalchemy.exc import IntegrityError

class TestAuthService(unittest.TestCase):
    def setUp(self):
        engine = create_engine("sqlite:///:memory:")
        Base.metadata.create_all(bind=engine)
        self.Session = sessionmaker(bind=engine)
        self.db = self.Session()

    def tearDown(self):
        self.db.close()

    def test_register_user_success(self):
        user = register_user(self.db, UserCreate(
            nome="Test", cognome="User", email="test@test.com", password="123456", ruolo=UserRole.RESPONSABILE))
        self.assertIsNotNone(user.id)

    def test_register_user_duplicate_email(self):
        register_user(self.db, UserCreate(
            nome="Test", cognome="User", email="dup@test.com", password="123456", ruolo=UserRole.RESPONSABILE))
        with self.assertRaises(ValueError):
            register_user(self.db, UserCreate(
                nome="Dup", cognome="User", email="dup@test.com", password="123456", ruolo=UserRole.RESPONSABILE))

    def test_authenticate_user_success(self):
        register_user(self.db, UserCreate(
            nome="Test", cognome="User", email="auth@test.com", password="123456", ruolo=UserRole.SEGRETERIA))
        user = authenticate_user(self.db, "auth@test.com", "123456")
```

```
        self.assertIsNotNone(user)

    def test_authenticate_user_wrong_password(self):
        register_user(self.db, UserCreate(
            nome="Test", cognome="User", email="wrong@test.com",
               password="123456", ruolo=UserRole.MAGAZZINIERE))
        user = authenticate_user(self.db, "wrong@test.com", "
           wrongpass")
        self.assertIsNone(user)

if __name__ == "__main__":
    unittest.main()
```

Test User Service

Controlla la gestione degli utenti lato back-end. Verifica che un utente possa essere creato, che possa essere eliminato e che il suo ruolo sia aggiornato correttamente nel database, mantenendo la coerenza dei dati.

```python
import unittest
from app.models.database import SessionLocal, Base, engine
from app.services.user_service import create_user, delete_user,
    get_all_users
from app.schemas.user_schema import UserCreate, UserRole

class TestUserService(unittest.TestCase):

    def setUp(self):
        Base.metadata.create_all(bind=engine)
        self.db = SessionLocal()
        self.db.query(get_all_users.__globals__['User']).delete()
        self.db.commit()

    def tearDown(self):
        self.db.close()

    def test_create_and_delete_user(self):
        user = create_user(self.db, UserCreate(
            nome="Mario",
            cognome="Bianchi",
            email="mario.bianchi@test.com",
            password="123456",
            ruolo=UserRole.SEGRETERIA
        ))
        self.assertIsNotNone(user.id)
        self.assertEqual(user.email, "mario.bianchi@test.com")

        delete_user(self.db, user.id)
        users = get_all_users(self.db)
        self.assertEqual(len(users), 0)

if __name__ == "__main__":
    unittest.main()
```

Test Product Service

Testa la corretta gestione dei prodotti. Verifica che i prodotti vengano creati senza errori e che il sistema restituisca l'elenco completo dei prodotti presenti nel database in modo coerente e aggiornato.

```python
import unittest
from app.models.database import SessionLocal, Base, engine
from app.services.product_service import create_product,
    get_product_by_id, get_all_products, update_product_quantity
from app.schemas.product_schema import ProductCreate
from app.models.product import Product

class TestProductService(unittest.TestCase):

    @classmethod
    def setUpClass(cls):
        # Creazione tabelle in un database temporaneo per i test
        Base.metadata.create_all(bind=engine)

    def setUp(self):
        # Pulizia database ad ogni test
        self.db = SessionLocal()
        self.db.query(Product).delete()
        self.db.commit()

    def tearDown(self):
        self.db.close()

    def test_create_product_with_potenza(self):
        """Verifica che un prodotto venga creato correttamente
            con la potenza (W)"""
        prodotto = create_product(
            self.db,
            ProductCreate(
                nome="Mixer Pro",
                categoria="Audio",
                quantita=10,
                modello="Pro 200",
                dimensione="Medio",
                brand="Yamaha",
                potenza=500
            )
        )
        self.assertIsNotNone(prodotto.id)
        self.assertEqual(prodotto.potenza, 500)

    def test_get_all_products_with_potenza(self):
        """Verifica che la potenza sia restituita correttamente
            tra tutti i prodotti"""
        create_product(
            self.db,
            ProductCreate(
```

```
                nome="Speaker Max",
                categoria="Audio",
                quantita=15,
                modello="Max 300",
                dimensione="Grande",
                brand="JBL",
                potenza=1200
            )
        )
        prodotti = get_all_products(self.db)
        self.assertGreater(len(prodotti), 0)
        self.assertTrue(any(p.potenza == 1200 for p in prodotti))

    def test_update_product_quantity_and_check_potenza(self):
        """Verifica che l'aggiornamento della quantit  non modifichi la potenza del prodotto"""
        prodotto = create_product(
            self.db,
            ProductCreate(
                nome="Amplificatore Lite",
                categoria="Audio",
                quantita=8,
                modello="Lite 100",
                dimensione="Piccolo",
                brand="Behringer",
                potenza=350
            )
        )
        update_product_quantity(self.db, prodotto.id, 20)
        prodotto_modificato = get_product_by_id(self.db, prodotto.id)

        self.assertEqual(prodotto_modificato.quantita, 20)
        self.assertEqual(prodotto_modificato.potenza, 350)

if __name__ == "__main__":
    unittest.main()
```

Test Rental Service

Assicura il corretto funzionamento della logica dei noleggi. Verifica la creazione di un nuovo noleggio con stato iniziale "in corso", la possibilità di concluderlo e il recupero di un noleggio specifico tramite il suo identificativo.

```python
import unittest
from datetime import date, timedelta
from app.models.database import SessionLocal, Base, engine
from app.models.rental import Rental
from app.models.product import Product
from app.schemas.rental_schema import RentalCreate
from app.services.rental_service import create_rental,
    update_rental, cancel_rental, conclude_rental

class TestRentalService(unittest.TestCase):
    def setUp(self):
        Base.metadata.create_all(bind=engine)
        self.db = SessionLocal()
        self.db.query(Rental).delete()
        self.db.query(Product).delete()
        self.db.commit()
        self.prodotto = Product(nome="Prodotto Test", categoria="
            Audio", quantita=10)
        self.db.add(self.prodotto)
        self.db.commit()
        self.db.refresh(self.prodotto)

    def tearDown(self):
        self.db.close()

    def test_create_rental_success(self):
        rental = create_rental(self.db, RentalCreate(
            prodotto_id=self.prodotto.id,
            quantita=2,
            cliente="Cliente Test",
            data_inizio=date.today(),
            data_fine=date.today() + timedelta(days=5),
            stato="in corso",
            metodo_pagamento="paypal"
        ))
        self.assertEqual(rental.quantita, 2)

    def test_create_rental_exceeding_quantity(self):
        with self.assertRaises(ValueError):
            create_rental(self.db, RentalCreate(
                prodotto_id=self.prodotto.id,
                quantita=50,
                cliente="Cliente Test",
                data_inizio=date.today(),
                data_fine=date.today() + timedelta(days=5),
                stato="in corso",
                metodo_pagamento="paypal"
```

```
            ))

    def test_update_rental_success(self):
        rental = create_rental(self.db, RentalCreate(
            prodotto_id=self.prodotto.id,
            quantita=1,
            cliente="Cliente Test",
            data_inizio=date.today(),
            data_fine=date.today() + timedelta(days=5),
            stato="in corso",
            metodo_pagamento="paypal"
        ))
        updated = update_rental(self.db, rental.id, quantita=2)
        self.assertEqual(updated.quantita, 2)

    def test_update_rental_exceeding_quantity(self):
        rental = create_rental(self.db, RentalCreate(
            prodotto_id=self.prodotto.id,
            quantita=1,
            cliente="Cliente Test",
            data_inizio=date.today(),
            data_fine=date.today() + timedelta(days=5),
            stato="in corso",
            metodo_pagamento="paypal"
        ))
        with self.assertRaises(ValueError):
            update_rental(self.db, rental.id, quantita=50)

    def test_cancel_rental(self):
        rental = create_rental(self.db, RentalCreate(
            prodotto_id=self.prodotto.id,
            quantita=1,
            cliente="Cliente Test",
            data_inizio=date.today(),
            data_fine=date.today() + timedelta(days=5),
            stato="in corso",
            metodo_pagamento="paypal"
        ))
        canceled = cancel_rental(self.db, rental.id)
        self.assertEqual(canceled.stato, "annullato")

    def test_conclude_rental(self):
        rental = create_rental(self.db, RentalCreate(
            prodotto_id=self.prodotto.id,
            quantita=1,
            cliente="Cliente Test",
            data_inizio=date.today(),
            data_fine=date.today() + timedelta(days=5),
            stato="in corso",
            metodo_pagamento="paypal"
        ))
```

```
        concluded = conclude_rental(self.db, rental.id)
        self.assertEqual(concluded.stato, "concluso")


if __name__ == "__main__":
    unittest.main()
```

Test Sale Service

Garantisce il corretto funzionamento della registrazione delle vendite. Controlla che le vendite vengano registrate con tutti i dati necessari e che sia possibile recuperarle correttamente tramite ID.

```python
import unittest
from datetime import date
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from app.models.database import Base
from app.models.product import Product
from app.services.sale_service import create_sale
from app.schemas.sale_schema import SaleCreate

class TestSaleService(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        """Configura un database in memoria per i test"""
        engine = create_engine("sqlite:///:memory:")
        TestingSession = sessionmaker(bind=engine)
        Base.metadata.create_all(bind=engine)
        cls.db = TestingSession()

        # Prodotto di base
        cls.product = Product(
            nome="Prodotto Vendita Test",
            categoria="Video",
            quantita=8
        )
        cls.db.add(cls.product)
        cls.db.commit()

    @classmethod
    def tearDownClass(cls):
        cls.db.close()

    def test_create_sale_success(self):
        """Verifica che una vendita valida venga creata
           correttamente"""
        sale = create_sale(self.db, SaleCreate(
            prodotto_id=self.product.id,
            quantita=3,
            cliente="Mario Rossi",
            data_vendita=date.today(),
            metodo_pagamento="paypal",
            stato="confermato"
        ))
        self.assertIsNotNone(sale.id)
        self.assertEqual(sale.quantita, 3)

    def test_create_sale_exceeding_quantity(self):
        """Verifica che non si possa vendere pi  dei disponibili
```

```
            """
        with self.assertRaises(ValueError):
            create_sale(self.db, SaleCreate(
                prodotto_id=self.product.id,
                quantita=50,  # maggiore dei disponibili
                cliente="Luca Bianchi",
                data_vendita=date.today(),
                metodo_pagamento="carta di credito",
                stato="confermato"
            ))

if __name__ == "__main__":
    unittest.main()
```

Test Damage Service

Controlla la gestione delle segnalazioni danni. Verifica che un danno venga registrato correttamente e che il conteggio dei danni per ogni prodotto sia preciso, permettendo un tracciamento corretto dello stato dei prodotti.

```python
import unittest
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from app.models.database import Base
from app.services.damage_service import report_damage,
    count_damages_for_product
from app.schemas.damage_schema import DamageCreate
from datetime import date

class TestDamageService(unittest.TestCase):
    def setUp(self):
        engine = create_engine("sqlite:///:memory:")
        Base.metadata.create_all(bind=engine)
        self.Session = sessionmaker(bind=engine)
        self.db = self.Session()

    def tearDown(self):
        self.db.close()

    def test_report_damage_success(self):
        damage = report_damage(self.db, DamageCreate(
            prodotto_id=1, descrizione="Danno test",
                data_segnalazione=date.today()))
        self.assertIsNotNone(damage.id)

    def test_count_damages_for_product(self):
        report_damage(self.db, DamageCreate(
            prodotto_id=1, descrizione="Danno", data_segnalazione
                =date.today()))
        count = count_damages_for_product(self.db, 1)
        self.assertEqual(count, 1)

if __name__ == "__main__":
    unittest.main()
```

Test Notification Service

Testa la logica delle notifiche. Verifica la corretta creazione delle notifiche, la possibilità di recuperarle tramite ID e di segnarle come lette, aggiornando coerentemente il database.

```python
import unittest
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from app.models.database import Base
from app.services.notification_service import create_notification, mark_notification_as_read, get_unread_notifications

class TestNotificationService(unittest.TestCase):
    def setUp(self):
        engine = create_engine("sqlite:///:memory:")
        Base.metadata.create_all(bind=engine)
        self.Session = sessionmaker(bind=engine)
        self.db = self.Session()

    def tearDown(self):
        self.db.close()

    def test_create_notification_success(self):
        notif = create_notification(self.db, messaggio="Test", tipo="info")
        self.assertEqual(notif.letto, 0)

    def test_mark_notification_as_read(self):
        notif = create_notification(self.db, messaggio="Test", tipo="info")
        mark_notification_as_read(self.db, notif.id)
        self.assertEqual(notif.letto, 1)

    def test_get_unread_notifications(self):
        create_notification(self.db, messaggio="Test", tipo="info")
        unread = get_unread_notifications(self.db)
        self.assertEqual(len(unread), 1)

if __name__ == "__main__":
    unittest.main()
```

Test Notification in Services

Questo ha lo scopo di verificare che ogni operazione rilevante effettuata all'interno del sistema generi correttamente una notifica associata. In particolare, il test controlla che la creazione, la modifica, la conclusione e l'annullamento di un noleggio, così come la registrazione e l'annullamento di una vendita, producano le notifiche attese nel database. Questo garantisce che il sistema mantenga sempre una cronologia accurata e aggiornata delle operazioni effettuate, migliorando la tracciabilità e la comunicazione tra gli operatori.

```

import unittest
from datetime import date, timedelta
from app.models.database import SessionLocal, Base, engine
from app.models.notification import Notification
from app.schemas.rental_schema import RentalCreate
from app.schemas.sale_schema import SaleCreate
from app.services.rental_service import (
    create_rental,
    update_rental,
    cancel_rental,
    conclude_rental
)
from app.services.sale_service import create_sale, cancel_sale
from app.services.notification_service import
   get_all_notifications
from app.schemas.product_schema import ProductCreate
from app.services.product_service import create_product


class TestNotificationsInServices(unittest.TestCase):

    def setUp(self):
        """Pulizia completa del database per ogni test"""
        Base.metadata.drop_all(bind=engine)
        Base.metadata.create_all(bind=engine)
        self.db = SessionLocal()

        # Creazione di un prodotto di test
        self.prodotto = create_product(self.db, ProductCreate(
            nome="TestProd",
            categoria="Audio",
            quantita=20
        ))

    def tearDown(self):
        self.db.close()

    def test_rental_notifications(self):
        """Verifica che tutte le operazioni sui noleggi generino
           notifiche"""
        r = create_rental(self.db, RentalCreate(
```

```
            prodotto_id=self.prodotto.id,
            quantita=2,
            cliente="Cliente1",
            data_inizio=date.today(),
            data_fine=date.today() + timedelta(days=5),
            stato="in corso",
            metodo_pagamento="contanti"
        ))

        update_rental(self.db, r.id, quantita=3)
        conclude_rental(self.db, r.id)
        cancel_rental(self.db, r.id)  # anche se gi  concluso, deve ignorarlo o notificare annullamento fallito

        notifs = [n.messaggio.lower() for n in get_all_notifications(self.db)]
        self.assertTrue(any("noleggio" in n and "registrato" in n for n in notifs))
        self.assertTrue(any("noleggio" in n and "modificato" in n for n in notifs))
        self.assertTrue(any("noleggio" in n and "concluso" in n for n in notifs))

    def test_sale_notifications(self):
        """Verifica che  l aggiunta  e  l annullamento  di una vendita generino notifiche"""
        s = create_sale(self.db, SaleCreate(
            prodotto_id=self.prodotto.id,
            quantita=1,
            cliente="ClienteVendita",
            data_vendita=date.today(),
            stato="confermato",
            metodo_pagamento="carta di credito"
        ))

        cancel_sale(self.db, s.id)
        notifs = [n.messaggio.lower() for n in get_all_notifications(self.db)]
        self.assertTrue(any("vendita" in n and "registrata" in n for n in notifs))
        self.assertTrue(any("vendita" in n and "annullata" in n for n in notifs))

    def test_auto_conclude_expired_rentals(self):
        """Verifica che i noleggi scaduti siano conclusi automaticamente"""
        r = create_rental(self.db, RentalCreate(
            prodotto_id=self.prodotto.id,
            quantita=1,
            cliente="Cliente2",
            data_inizio=date.today() - timedelta(days=10),
```

```
            data_fine=date.today() - timedelta(days=5),
            stato="in corso",
            metodo_pagamento="contanti"
        ))

        # Chiama manualmente la funzione che aggiorna i noleggi
            scaduti
        conclude_rental(self.db, r.id)
        self.db.refresh(r)

        self.assertEqual(r.stato, "concluso")


if __name__ == "__main__":
    unittest.main()
```